Universidade Federal do Piauí
Centro de Educação Aberta e a Distância

LABORATÓRIO DE PROGRAMAÇÃO

José Ricardo Mello Viana



Ministério da Educação - MEC
Universidade Aberta do Brasil - UAB
Universidade Federal do Piauí - UFPI
Centro de Educação Aberta e a Distância - CEAD

Laboratório de Programação

José Ricardo Mello Viana

UFPI
2010

PRESIDENTE DA REPÚBLICA	*Dilma Vana Rousseff Linhares*
MINISTRO DA EDUCAÇÃO	*Aloizio Mercadante*
GOVERNADOR DO ESTADO	*Wilson Nunes Martins*
REITOR DA UNIVERSIDADE FEDERAL DO PIAUÍ	*José Arimatéia Dantas Lopes*
PRESIDENTE DA CAPES	*Jorge Almeida Guimarães*
COORDENADOR GERAL DA UNIVERSIDADE ABERTA DO BRASIL	*João Carlos Teatini de S. Clímaco*
DIRETOR DO CENTRO DE EDUCAÇÃO ABERTA E A DISTÂNCIA DA UFPI	*Gildásio Guedes Fernandes*

COORDENADORES DE CURSOS

ADMINISTRAÇÃO	*Antonella Maria das Chagas Sousa*
ADMINISTRAÇÃO PÚBLICA	*Fabiana Rodrigues de Almeida Castro*
CIÊNCIAS BIOLÓGICAS	*Maria da Conceição Prado de Oliveira*
FILOSOFIA	*Zoraida Maria Lopes Feitosa*
FÍSICA	*Miguel Arcanjo Costa*
LETRAS PORTUGUÊS	*José Vanderlei Carneiro*
LETRAS INGLÊS	*Lívia Fernanda Nery da Silva*
MATEMÁTICA	*João Benício de Melo Neto*
PEDAGOGIA	*Vera Lúcia Costa Oliveira*
QUÍMICA	*Milton Batista da Silva*
SISTEMAS DE INFORMAÇÃO	*Leonardo Ramon Nunes de Sousa*

EQUIPE DE DESENVOLVIMENTO

TÉCNICOS EM ASSUNTOS EDUCACIONAIS	*Zilda Vieira Chaves*
	Ubirajara Santana Assunção
	Djane Oliveira de Brito
EDIÇÃO	*Roberto Denes Quaresma Rêgo*
PROJETO GRÁFICO	*Samuel Falcão Silva*
DIAGRAMAÇÃO	*Samuel Falcão Silva*
REVISÃO ORTOGRÁFICA	*Maria da Penha Feitosa*
REVISÃO GRÁFICA	*Carmem Lúcia Portela Santos*

CONSELHO EDITORIAL DA EDUFPI

Prof. Dr. Ricardo Alaggio Ribeiro (Presidente)
Des. Tomaz Gomes Campelo
Prof. Dr. José Renato de Araújo Sousa
Profª. Drª. Teresinha de Jesus Mesquita Queiroz
Profª. Francisca Maria Soares Mendes
Profª. Iracildes Maria de Moura Fé Lima
Prof. Dr. João Renór Ferreira de Carvalho

V614L Viana, José Ricardo Mello
Laboratório de Programação/ FJosé Ricardo Mello Viana.--- Teresina: EDUFPI, 2010.
80 p.

1- Programação. 2 - Algoritmos. I. Título

C.D.D. 001.5

© 2010. Universidade Federal do Piauí - UFPI. Todos os direitos reservados.

A responsabilidade pelo conteúdo e imagens desta obra é do autor. O conteúdo desta obra foi licenciado temporária e gratuitamente para utilização no âmbito do Sistema Universidade Aberta do Brasil, através da UFPI. O leitor se compromete a utilizar oconteúdo desta obra para aprendizado pessoal, sendo que a reprodução e distribuição ficarão limitadas ao âmbito interno dos cursos. A citação desta obra em trabalhos acadêmicos e/ou profissionais poderá ser feita com indicação da fonte. A cópia deste obra sem autorização expressa ou com intuito de lucro constitui crime contra a propriedade intelectual, com sansões previstas no Código Penal.

Apresentação

Nosso objetivo neste livro é pôr em prática os conceitos vistos em Algoritmos e Programação I. Durante toda o livro teremos muitos exemplos, exercícios resolvidos e exercícios propostos com o objetivo de estimular você a colocar “a mão na massa” ao implementar os algoritmos descritos.

Todas as atividades aqui propostas devem ser realizadas, a fim de propiciar a você cada vez mais confiança na resolução de problemas através de algoritmos computacionais, constituindo-se, assim, uma disciplina de fundamental importância na continuidade do curso de Bacharelado em Sistemas de Informação.

Nossa primeira unidade está voltada para os conceitos iniciais, como preparação do ambiente de programação (Eclipse + MinGW ou GNU C) e primeiros exemplos de entrada e saída, utilizando a linguagem de programação C. Logo em seguida, na unidade 2, iremos pôr em prática os comandos de seleção e de repetição. Na unidade 3, revisaremos os conceitos de vetores e matrizes, formas de declaração que nos permitem criar vários elementos do mesmo tipo, sequencialmente. A unidade 4 aborda o conceito de estruturas, como declará-las e a melhor forma de utilizá-las em nossos programas. Por fim, na unidade 5, colocaremos em prática os conceitos de funções e procedimentos, como forma de modularizar nosso programa e facilitar o entendimento e implementação do mesmo.

Este material foi preparado visando abordar estes conceitos da melhor forma possível, sempre pensando no aluno à distância e nesta particular forma de ensino. Procuramos colocar os exemplos de forma bastante explicativa, a fim de que você possa implementar os exemplos e resolver os exercícios mesmo na ausência de Internet para eventuais pesquisas.

Então, é isso. Aproveite o livro para praticar bastante. Mãos à obra.

Bons Estudos!

Sumário

09 **UNIDADE 1**
PRIMEIROS PASSOS

Instalação do Compilador C 11
Instalação do Eclipse 16
Primeiros passos no ambiente de programação 17
Criando um projeto 18
Entendendo o projeto criado 19
Estrutura básica de um programa C 21
Entrada de dados 23
Declaração de variáveis 26
Atribuição de valores a variáveis 27

35 **UNIDADE 2**
COMANDOS DE SELEÇÃO E DE REPETIÇÃO

Comandos de Seleção 35
Comandos de Repetição 52

57 **UNIDADE 3**
VETORES E MATRIZES

Vetores 73
Strings ou Cadeias de caracteres (Vetores de char) 78
Matrizes 84

09

UNIDADE 4
ESTRUTURAS

Estruturas 95
Declaração de tipos de dados estruturas 97
Vetores de Estruturas 101
Estruturas de Estruturas 104

09

UNIDADE 5
FUNÇÕES E PROCEDIMENTOS

Funções 111
Criando Funções 112
Procedimentos 114
Passagem de parâmetros 116
Protótipo de funções 126
Recursividade 128

UNIDADE 01

Primeiros Passos

1 PRIMEIROS PASSOS

Primeiros passos

Antes de começarmos a programar na linguagem C, devemos preparar o ambiente de programação que servirá de apoio no desenvolvimento de nossos aplicativos. O ambiente escolhido será o Eclipse (sua versão para C/C++) e o compilador de C (para Windows o MinGW e para Linux o GNU C).

Sendo assim, nas próximas sessões serão mostradas as instalações e configurações desse ambiente nessas duas plataformas (Windows e Linux).

Instalação do Compilador C

Primeiramente, vamos aprender a como instalar o compilador da linguagem que utilizaremos durante o curso (C). Iremos mostrar a instalação dos compiladores para os dois ambientes, Windows e Linux, respectivamente, nesta ordem.

Instalação do MinGW para Windows

O MinGW (Minimalist GNU for Windows) é o compilador C/C++ que iremos utilizar para desenvolvermos algoritmos em C para Windows. Podemos baixar a versão mais recente do MinGW no seguinte endereço:

http://ufpr.dl.sourceforge.net/sourceforge/mingw/MinGW-5.1.4.exe

Para instalarmos o MinGW seguiremos os seguintes passos especificados nas telas mostradas nas figuras a seguir:

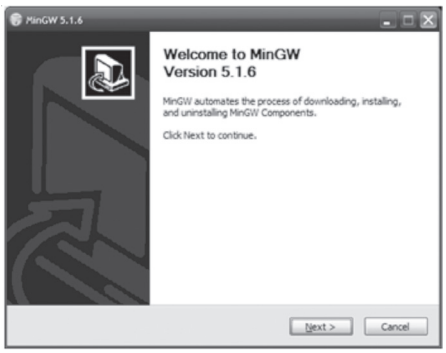


Figura 1. Tela inicial de instalação do MinGW

A **Figura 1** mostra a tela inicial de instalação do MinGW. A partir dela vamos continuar com nossa instalação, pressionando no botão “next”.



Figura 2. Instalação do MinGW: Modo de download

Ao pressionar “next” somos redirecionados à tela da **Figura 2**, onde teremos que escolher se iremos baixar e instalar (Download and install) o MinGW ou apenas baixar (Download only). Por padrão, deixamos selecionada a primeira opção e pressionamos “next” novamente.



Figura 3. Instalação do MinGW: licença de uso

Prosseguindo, veremos a tela da Figura 3, que mostra a licença de uso do MinGW. Para continuarmos a instalação, devemos aceitar os termos da licença clicando em aceito (I agree).



Figura 4. Instalação do MinGW: Pacote a ser instalado

Após aceitar a licença de uso, veremos a tela da **Figura 4**. Nela devemos escolher qual versão do MinGW iremos instalar: versão prévia (previous), corrente (current) ou candidata (candidate). Aceitaremos a sugestão dada por padrão e escolhemos a opção corrente pressionando no botão “next”.

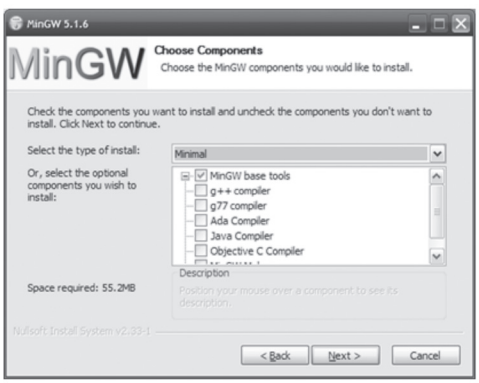


Figura 5. Instalação do MinGW: escolha de componentes

Após escolhermos a versão seremos redirecionados para a tela da **Figura 5**, na qual devemos escolher quais componentes serão instalados. Por padrão, é selecionada a opção “MinGW base tools”, que instalará todo o ferramental necessário para desenvolvermos em C. Podemos selecioná-la e continuarmos pressionando “next”.

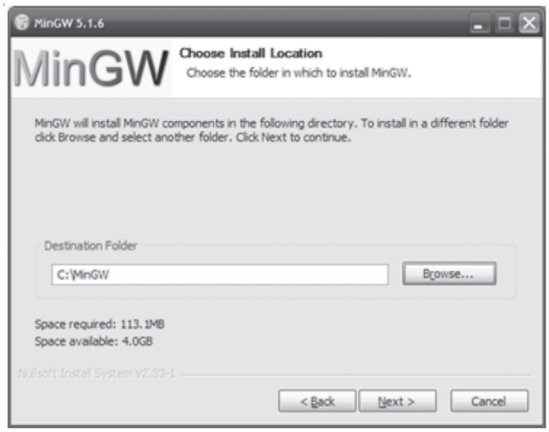


Figura 6. Instalação do MinGW: Escolha da pasta

Na tela acima (Figura 6) devemos escolher a pasta onde será instalado o compilador. Basta escolhermos uma pasta qualquer, ou aceitarmos a sugestão do instalador (C:\MinGW) e continuarmos pressionando “next”.



Figura 7. Finalização da instalação do MinGW

Nesta última tela (Figura 7) escolhemos onde será colocado o MinGW no menu principal do Windows. Basta escolhermos uma opção ou aceitarmos a sugestão do instalador (MinGW) e pressionarmos Install.

Nesse momento, o instalador irá baixar da Internet os arquivos necessários para o correto funcionamento do MinGW. Após baixar todos os arquivos, o compilador será instalado na pasta escolhida e será mostrada a tela a seguir (Figura 8).



Figura 8. Instalação do MinGW completada

Esta tela (Figura 8) confirma que a instalação foi completada com sucesso e que estamos aptos a utilizar nosso compilador C para a criação de algoritmos. Ao clicarmos em “next” somos redirecionados para a tela de finalização a seguir (Figura 9).

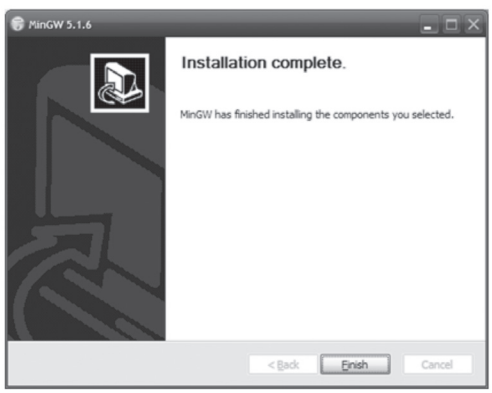


Figura 9. Finalização da instalação do MinGW

Instalação do GNU C para Linux

Para instalarmos o compilador GNU C em ambiente Linux, basta executarmos o seguinte comando em um terminal:

sudo apt-get install gcc

Feito isso, o Linux se encarregará de baixar da Internet os arquivos necessários e instalá-los em nosso computador. Será solicitada uma senha para a instalação do aplicativo, quando devemos informar a

mesma senha usada para efetuar *login* no sistema. (Em alguns ambientes Linux o compilador C vem instalado por padrão. Para verificarmos isso basta tentar executar o comando gcc em um terminal. Se o seu Linux reconhecer o comando e retornar um erro do tipo: *no input files*, significa que você já o possui, caso contrário execute o comando mostrado anteriormente).

Instalação do Eclipse

O Eclipse é um Ambiente para Desenvolvimento Integrado (Integrated Development Environment - IDE) disponível para várias linguagens de programação diferentes. Em nosso caso, como estamos utilizando C, iremos baixar a versão para esta linguagem. No site oficial do eclipse (www.eclipse.org) podemos encontrar a versão específica para desenvolvimento em C/C++ no endereço a seguir:

http://www.eclipse.org/downloads/packages/eclipse-ide-cc-developers/heliosr

Nesta página podemos encontrar na barra lateral direita os endereços para baixarmos versões específicas para cada sistema operacional. Seguem os endereços específicos:

1) **Windows 32 bits:**
http://www.eclipse.org/downloads/download.php?file=/technology/epp/downloads/release/helios/R/eclipse-cpp-helios-win32.zip
2) **Windows 64 bits:**
http://www.eclipse.org/downloads/download.php?file=/technology/epp/downloads/release/helios/R/eclipse-cpp-helios-win32-x86_64.zip
3) **Linux 32 bits:**
http://www.eclipse.org/downloads/download.php?file=/technology/epp/downloads/release/helios/R/eclipse-cpp-helios-linux-gtk.tar.gz
4) **Linux 64 bits:**
http://www.eclipse.org/downloads/download.php?file=/technology/epp/downloads/release/helios/R/eclipse-cpp-helios-linux-gtk-x86_64.tar.gz

Após baixarmos a versão específica de nosso sistema operacional, basta extrair o arquivo em uma pasta específica de nosso computador. A pasta eclipse será criada e dentro dela estarão os executáveis do eclipse (OBS.: como é feito em Java, é necessário termos o Java instalado para podermos executar o eclipse).

Feito isso, estamos com tudo pronto para começarmos a criar nossos primeiros programas em C usando o ambiente que acabamos de instalar.

Primeiros passos no ambiente de programação

Ao executarmos o eclipse, seremos questionados sobre a pasta na qual será salvo nosso workspace, como mostra a Figura 10 a seguir:

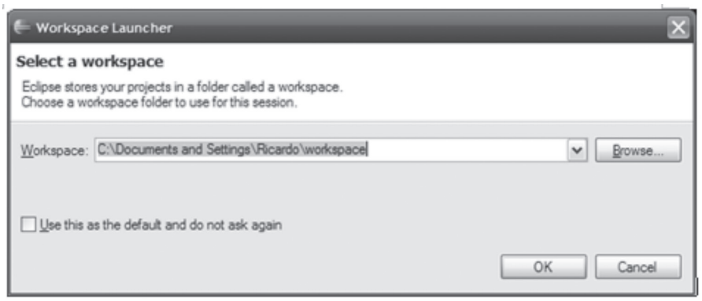


Figura 10. Seleção de workspace no Eclipse

Nessa tela (Figura 10) devemos selecionar a pasta de nossa preferência para o eclipse gravar nossos projetos e clicarmos no botão “ok”. Após isso, o eclipse será carregado e a janela a seguir (Figura 11) será aberta:

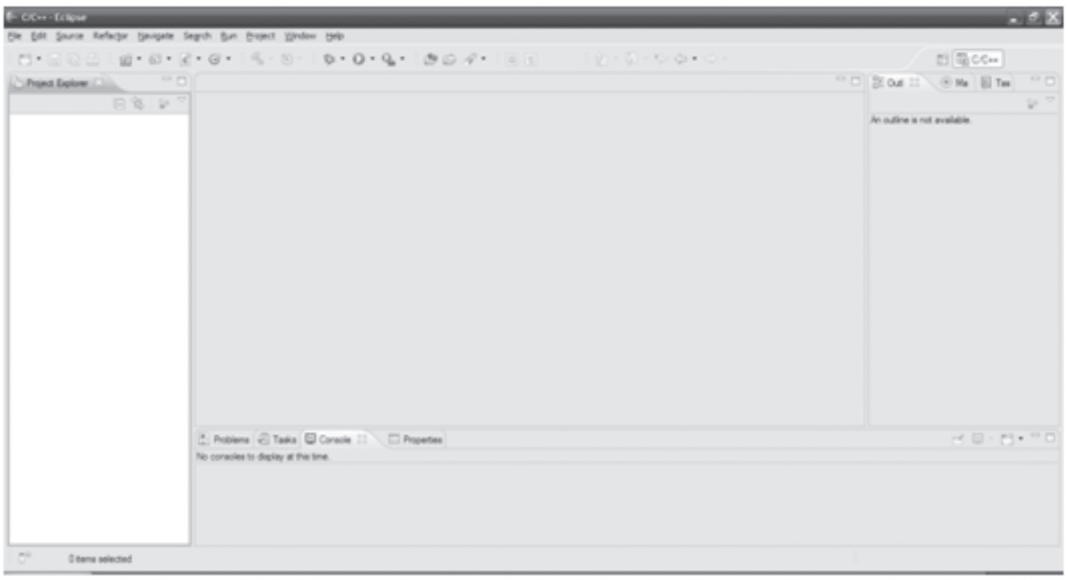


Figura 11. Tela inicial do eclipse

Se tudo correu bem, estamos aptos a criar nosso primeiro algoritmo em C usando o ambiente Eclipse + MinGW ou GCC.

Criando um projeto

Antes de codificarmos nossos algoritmos propriamente ditos, devemos dar atenção à forma como nossa IDE trabalha. O eclipse organiza nosso código em projetos, portanto, antes de criarmos um arquivo fonte devemos criar e configurar um projeto no nosso ambiente de programação. Para criar um projeto C no eclipse devemos seguir os seguintes passos.

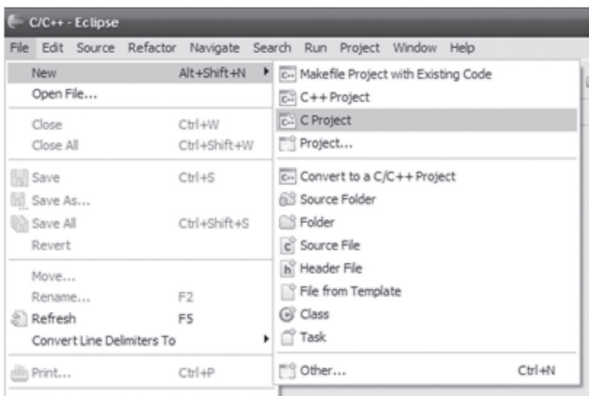


Figura 12. Criação de um novo projeto

Começamos selecionando o menu “File -> New -> C-Project”, através do qual informamos à IDE qual tipo de projeto queremos criar. Fazendo isso, a janela a seguir (Figura 13) nos será mostrada.

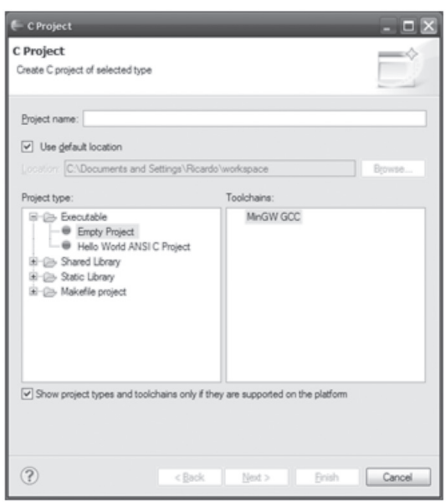


Figura 13. Dados do novo projeto

Nessa janela (Figura 13) vamos configurar nosso projeto. Primeiramente, devemos dar um nome a nosso projeto, no campo *Project name*. Na localização do projeto, o eclipse já sugere uma pasta com o mesmo nome que colocamos no projeto dentro do *workspace* que escolhemos. Iremos aceitar essa sugestão de modo a organizar nossos projetos.

Devemos selecionar também o tipo de projeto. Por padrão, está selecionado *Empty Project*, que significa projeto vazio, no entanto, utilizaremos a opção *Hello World ANSI C Project* para criação do nosso projeto. Esta opção cria um arquivo com a implementação do clássico *Hello World*, que simplesmente mostra uma mensagem de olá na tela. O campo de seleção *Toolchains* só possui a opção MinGW GCC, pois só temos este compilador instalado neste caso.

Feito isso, clicamos no botão *Finish* e nosso primeiro projeto será criado. A seguinte tela (Figura 14) será mostrada.

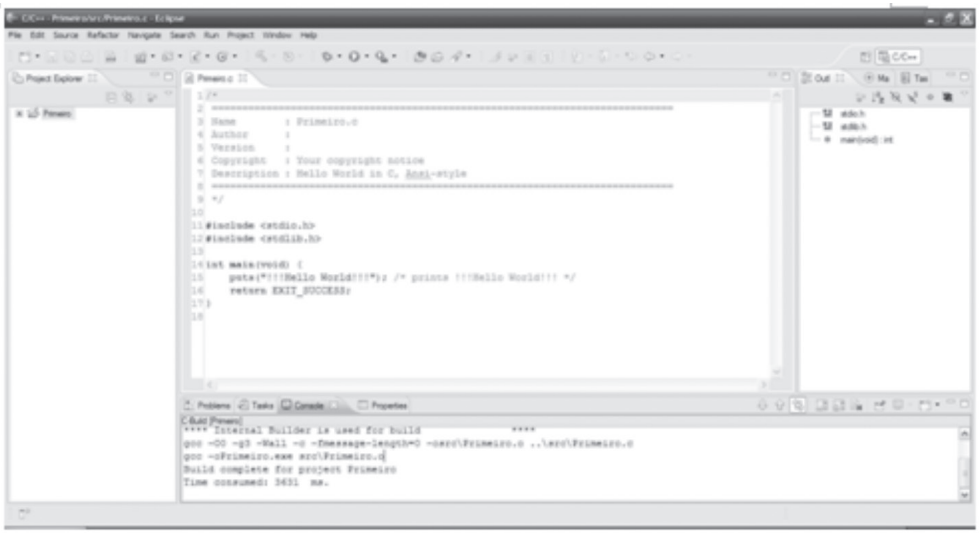


Figura 14. Projeto criado com sucesso

Entendendo o projeto criado

Ao criarmos o projeto, o seguinte arquivo foi criado e aberto automaticamente para edição no eclipse (Figura 15):



Figura 15. Arquivo criado automaticamente pelo eclipse

Neste arquivo são usados os comandos básicos de C para implementar um algoritmo que escreva a mensagem "!!!Hello World!!!" na tela. Detalhando cada linha, temos o conteúdo a seguir:

Linhas 1 a 9: Comentário de várias linhas (Aberto com /* e fechado com */);
Linhas 11 e 12 – #include<stdlib.h> e #include<stdio.h>: Inclusão das bibliotecas stdio e stdlib, bibliotecas básicas para entrada e saída de dados em C (utilização das funções printf e scanf principalmente);
Linha 14 – int main(void) {: Declaração da função principal e abertura do bloco de comandos correspondentes;
Linha 15 – puts("!!!Hello World!!!");: Função para saída de dados (puts tem a mesma função que printf) seguida de um comentário;
Linha 16 – return EXIT_SUCCESS;: Retorno da função principal (EXIT_SUCCESS corresponde à constante 0);
Linha 17 – }: Corresponde ao fechamento do bloco de comandos da função principal (finalização do nosso algoritmo).

Para que possamos executar nosso programa, primeiro é necessário compilá-lo, ou seja, gerar o executável correspondente ao código fonte digitado. Via eclipse podemos fazer isso de duas maneiras: clicando no menu *Project* e selecionando a opção *Build All* ou através do atalho de teclado "Ctrl+B". Depois de compilado podemos executar nosso programa através do botão da barra de ferramentas destacado na figura a seguir (Figura 16).

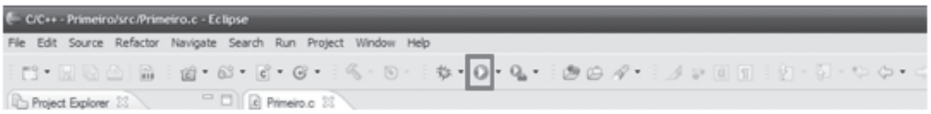


Figura 16. Executar projeto no eclipse

Feito isso, o resultado da execução será mostrado na parte de baixo da IDE, na aba console (aberta automaticamente quando da execução de nosso algoritmo). Podemos verificar o resultado da execução deste exemplo na figura a seguir (Figura 17):

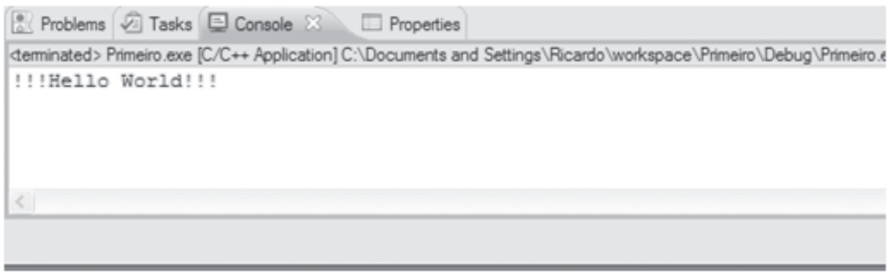


Figura 17. Execução do projeto

Como era de se esperar, nosso programa escreve, no console, a mensagem: “!!!Hello World!!!” e não acontecem erros.

Estrutura básica de um programa C

Neste momento, passaremos a nos focar nos algoritmos e sua implementação em C, utilizando a ferramenta exposta (eclipse) para desenvolvê-los. A partir do projeto criado anteriormente, apagaremos o conteúdo do arquivo mostrado na **Figura 15** e, em seu lugar, digitaremos o seguinte conteúdo:

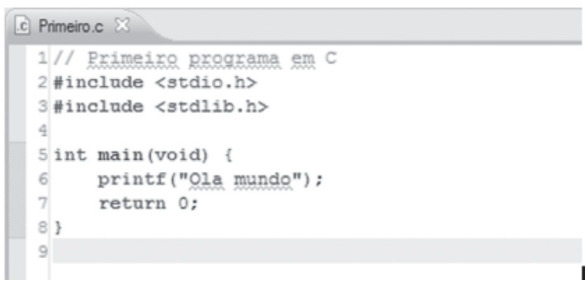


Figura 18. Primeiro exemplo: Olá mundo

A rigor, este exemplo é bem parecido com o que foi mostrado anteriormente. No entanto, está mais condizente com o que vimos em nossa disciplina anterior de Algoritmos e Programação I. A explicação, linha por linha, deste código está logo abaixo. Temos apenas um comando de saída de dados. Vejam que é bastante semelhante ao exemplo anterior.

Linha 1 – //Primeiro programa em C: Comentário de uma linha (a partir do // até o fim da linha);
Linhas 2 e 3 – #include<stdlib.h> e #include<stdio.h>: Inclusão de bibliotecas stdlib e stdio para operações de entrada e saída;
Linha 5 – int main(void) {: Declaração da função main e abertura do programa;
Linha 6 – printf("Ola mundo");: Função printf, responsável por escrever a mensagem "Olá mundo" na tela;
Linha 7 – return 0;: Retorno da função principal (0);
Linha 8 – }: Fechamento do corpo da função principal (finalização do programa).

Lembra disso?

A função printf, usada nesse exemplo, serve como interface com o mundo externo. É através dela que damos saída aos dados em nossos programas. Ela possui a seguinte sintaxe: *printf* (texto a ser escrito, variável <ou variáveis> para preenchimento).

Podemos passar códigos como parâmetros para o texto esta função, que serão substituídas pelas variáveis ou valores passados como parâmetro logo em seguida. Por exemplo:

Código	Tipo de variável
%d	Inteiro (int)
%f	Real (float)
%c	Caractere (char)
%s	Cadeia de caracteres (char[])

Tabela 1. **Códigos para a função** *printf*

É possível, ainda, passar parâmetros capazes de imprimir caracteres especiais, necessários, especialmente, para formatarmos a

saída do nosso programa. Por exemplo: \n (quebra de linha), \t (tabulação), \\ (escreve a barra), entre outros.

Exercícios Propostos

1. Faça um programa que escreva seu nome.
2. Faça um programa que desenhe um coração usando o caractere * para fazer o contorno.
3. Faça um programa que escreva suas notas em linhas diferentes com exatamente uma casa decimal. Use a formatação %.1f para substituir pelos valores e, a partir disso, explique como esse marcador funciona.
4. (Resolvido) Faça um programa que escreva seu nome e sobrenomes, cada um em uma linha.

Laboratorio.c

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 int main() {
5     printf("José\n");
6     printf("Ricardo\n");
7     printf("Mello\n");
8     printf("Viana\n");
9     return 0;
10 }
```

Execução:

Console

```
<terminated> Laboratorio
José
Ricardo
Mello
Viana
```

Entrada de dados

Da mesma forma que precisamos dar algum tipo de retorno para que o usuário saiba o que está acontecendo em nosso programa (função printf), muitas vezes precisamos que o usuário interaja com o programa informando dados para que sejam usados nos cálculos que o programa

irá executar. Para isso, usaremos a função scanf, que possui sintaxe bem parecida com o printf e serve para lermos um valor do teclado e armazenarmos em uma variável.

scanf (modo de leitura, endereço da variável <ou variáveis>)

Para informar o modo de leitura das variáveis usamos os mesmos códigos já vistos para o printf. Para informar o endereço da variável, não devemos nos esquecer de colocar o caractere & antes do nome cada uma das variáveis. É ele quem diz ao compilador que quero passar o endereço de memória da variável, e não o valor da variável. Vejamos o exemplo a seguir (Figura 19):

principal.c

```
1  // Leitura de valores com scanf
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void) {
6      int idade;
7      printf("Qual a sua idade? ");
8      fflush(stdout);
9      scanf("%d", &idade);
10     printf("Sua idade é %d", idade);
11     return 0;
12 }
13
```

Figura 19. Exemplo de *scanf*

A seguir a explicação, linha por linha, deste exemplo:

Linha 1 – //Leitura de valores com scanf: Comentário sobre o programa;
Linhas 2 e 3 – #include<stdlib.h> e #include<stdio.h>: Inclusão de bibliotecas de entrada e saída;
Linha 5 – int main(void) {: Declaração da função principal e abertura do bloco do programa;
Linha 6 – int idade;: Declaração de uma variável (explicação logo a seguir);
Linha 7 – printf(“Qual a sua idade? ”);: Saída de uma mensagem para o usuário;

Linha 8 – fflush(stdout);: Comando que “manda” escrever imediatamente na tela;
Linha 9 – scanf(“%d”, &idade);: Leitura de um valor inteiro (%d) do teclado e armazenamento em idade;
Linha 10 – printf(“Sua idade é %d”, idade);: Escrita de uma mensagem de retorno com o valor da variável lida;
Linha 11 – return 0;: Retorno da função principal;
Linha 12 – }: Fechamento do corpo da função principal.

Podemos verificar a saída deste programa na **Figura 20** a seguir:

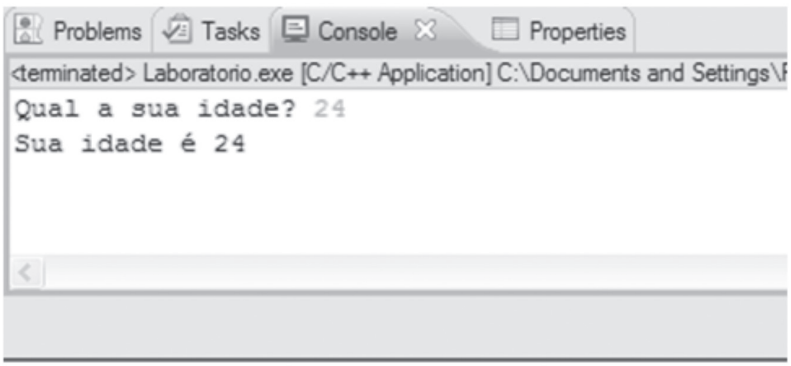


Figura 20. Resultado da execução

OBS.: Neste momento, cabe uma observação importante. Na **linha 8** tivemos que usar um comando novo *(fflush(stdout)*), que faz com que a saída anterior seja escrita imediatamente. Só foi necessário usar esse comando devido a um problema com a janela de console do eclipse para ambiente Windows que não mostra todas as saídas até que as entradas sejam lidas. Vejam, na **Figura 21**, como seria a execução do programa sem o comando *fflush:*

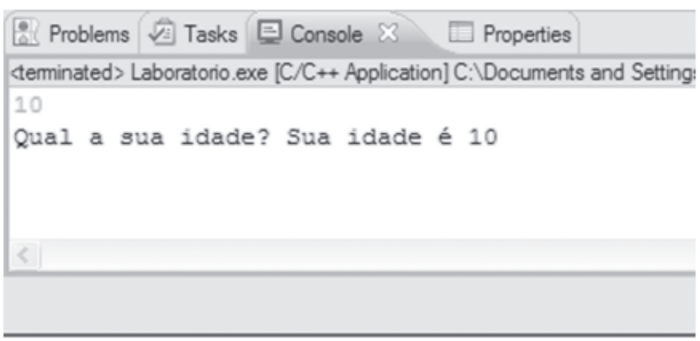


Figura 21. Execução do programa sem *fflush*

Vale ressaltar que este problema só ocorre no console do eclipse para Windows, portanto, se estivermos executando nossos exemplos em ambiente Linux ou em outros terminais que não o do eclipse para Windows, este comando se faz desnecessário. Portanto, se quisermos usar o console do eclipse para Windows devemos sempre colocar o comando fflush(stdout) após os comandos de escrita (printf) em nossos programas.

Declaração de variáveis

Muitas vezes é necessário armazenar algum valor durante a execução de nossos programas. Valores esses que, possivelmente, serão usados mais à frente em algum cálculo. Para isso, usamos o conceito de variáveis: uma variável é uma posição da memória do computador onde podemos armazenar um valor para utilização no nosso programa. Para usarmos uma variável, primeiramente precisamos declará-la, ou seja, informar ao computador que aquela variável existe. O formato de declaração de variáveis em C é:

tipo_da_variável nome_da_variável; ou
tipo_da_variável nome_variável_1, nome_variável_2, ..., nome_variável_n;

Podemos declarar apenas uma variável por declaração ou aproveitar o tipo que estamos utilizando para declarar, ao mesmo tempo, várias variáveis pertencentes a esse tipo. Ao especificarmos o tipo de uma variável estamos selecionando o conjunto de valores que esta variável pode assumir, as operações que podem ser efetuadas com ela e, ainda, o formato de representação para a variável.

Vale lembrar que, diferente de outras linguagens, bool não existe por padrão na linguagem C como tipo básico. Para utilizarmos esse tipo devemos incluir a biblioteca stdbool.h através da chamada:

#include<stdbool.h>

Restrições para nomes

Para especificação de nomes de variáveis devemos seguir algumas restrições, tais como:

a) Não se deve iniciar com número. Ex: 1var (inválida);

b) Não se devem utilizar caracteres especiais. Ex: var*, var#, média (inválidas);
c) Não se pode colocar espaço ou hífen. Ex: valor total, valor total (inválidas);
    i. Utilize, nesses casos, underline. **Ex**: valor_total
d) Procure utilizar nomes que lembrem o significado da variável. **Ex:** para uma variável que irá armazenar a média: float media.

É importante destacarmos, ainda, que a linguagem C é case sensitive (sensível ao caso). Ou melhor, há diferença entre maiúsculas e minúsculas. Sendo assim, a variável total é diferente da variável TOTAL para a linguagem C.

Atribuição de valores a variáveis

Para atribuirmos um valor a uma variável utilizamos o operador '='. Dessa forma, podemos armazenar um valor em uma variável previamente declarada. É possível, ainda, atribuir um valor a uma variável no momento de sua declaração.

Para usar o valor que foi atribuído a uma variável basta utilizarmos o próprio nome da variável nos cálculos posteriores. Vejamos o exemplo a seguir (Figura 22):

Primeiro.c

```
1  // Declaração de variáveis e atribuição de valor
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void) {
6      int a, b;
7      a = 10;
8      b = 18;
9      int soma = a + b;
10     printf("Soma de %d + %d é %d", a, b, soma);
11     return 0;
12 }
13
```

Figura 22. Exemplo de declaração e utilização de variáveis

Neste exemplo utilizamos os conceitos mostrados anteriormente de declaração de variáveis, atribuição de valores a variáveis e, ainda, atribuição de um valor a uma variável no momento da declaração da

mesma. Vejamos a explicação desse código linha por linha:

Linha 1 – //Declaração de variáveis e atribuição de valor: Comentário sobre o programa;
Linhas 2 e 3 – #include<stdio.h> e #include<stdlib.h>: Inclusão de bibliotecas para entrada e saída;
Linha 5 – int main(void) {: Declaração da função main e abertura do corpo da mesma;
Linha 6 – int a, b;: Declaração de duas variáveis do tipo inteiro (int);
Linhas 7 e 8 – a = 10; e b = 18;: Atribuição de valores às variáveis a e b (valores devem ser inteiros);
Linha 9 – int soma = a + b;: Declaração de uma nova variável e atribuição do resultado da soma de a e b;
Linha 10 – printf("Soma de %d + %d é %d", a, b, soma);: Escrita da saída do programa (Soma de 10 + 18 é 28);
Linha 11 – return 0;: Retorno da função principal;
Linha 12 – }: Fechamento do corpo da função principal.

O resultado da execução deste programa pode ser visto na Figura 23, a seguir:

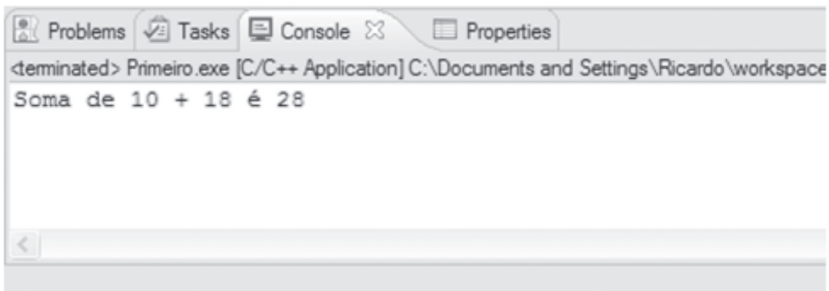


Figura 23.A Resultado da execução

Expressões

Podemos criar expressões aritméticas relacionais e lógicas usando os respectivos operadores que a linguagem disponibiliza. Vejamos alguns deles:

Operador	Significado
+	Soma
-	Subtração
*	Multiplicação
/	Divisão (inteira ou real)
%	Resto da divisão inteira

Tabela 3. Operadores aritméticos

Operador	Símbolo
Igual	=
Maior	>
Menor	<
Maior ou Igual	>=
Menor ou Igual	<=
Diferente	|=

Tabela 4. Operadores relacionais

Operador	Símbolo
Conjunção - E	&&
Disjunção - OU	II
Negação - NÃO	!

Tabela 5. Operadores lógicos

Exercícios Propostos



1. Faça um programa que leia um número nome e escreva a mensagem “O número digitado foi” seguido do número recebido: Ex: entrada: 10; saída: O número digitado foi 10.
2. (Resolvido) Faça um programa que receba dois números, calcule e imprima o resultado da divisão inteira e o resto da divisão inteira entre eles.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>

int main() {
    int pri, seg;
    printf("Digite o primeiro número: ");
    fflush(stdout);
    scanf("%d", &pri);
    printf("Digite o segundo número: ");
    fflush(stdout);
    scanf("%d", &seg);
    int div = pri/seg;
    int resto = pri % seg;
    printf("Divisão: %d\n", div);
    printf("Resto: %d\n", resto);
    return 0;
}
```

Execução:

Console

```
<terminated> Laboratorio.exe [C/C++ Applica
Digite o primeiro número: 10
Digite o segundo número: 3
Divisão: 3
Resto: 1
```

3. Faça um programa que crie três variáveis para receber as três notas de um aluno e calcule e imprima a média desse aluno.
4. Faça um programa que receba sua idade e escreva sua idade daqui a 10 anos.
5. Faça um programa que calcule o resultado da expressão: (537 - 285) * 10 + (3 * (72 - 17)).
6. Faça um programa que receba um número inteiro e calcule o resto da divisão por 2.
7. Faça um programa que receba o nome e o sobrenome de uma pessoa e imprima o nome completo em uma linha.
8. Faça um programa que receba um número e exiba seu sucessor e antecessor.
9. Faça um programa que calcule a quantidade de dinheiro gasto por um fumante dados: a quantidade de anos que ele fuma, o preço da carteira e o número de cigarros por dia.
10. Faça um programa que calcule a área e o volume do cilindro.
11. Faça um programa que resolva uma equação do primeiro grau (ax + b = 0).
12. Faça um programa que receba uma temperatura em Celcius e imprima

a correspondente conversão para Farenheit e Kelvin.

13. Sabendo que latão é composto por 70% de cobre e 30% de zinco, faça um programa que calcule a quantidade de cobre e zinco para uma determinada quantidade de latão informada pelo usuário.

14. Faça um programa que imprima uma caixa, uma oval, uma seta e um losango usando asteriscos da seguinte forma:



Resumindo

Nessa unidade preparamos, primeiramente, nosso ambiente de programação. Vimos como instalar nossa IDE, o Eclipse, e como preparar o compilador para executar nossos primeiros exemplos na linguagem de programação C. Percebendo a diversidade de alunos e de polos que íamos encontrar, essa unidade dispôs de instruções para preparação do ambiente tanto para o sistema operacional Windows como para o sistema operacional Linux. Revimos também exemplos simples de C para praticar os comandos de entrada e saída, *printf* e *scanf*.

UNIDADE 02

Comandos de Seleção e de Repetição

2 COMANDOS DE SELEÇÃO E DE REPETIÇÃO

Normalmente, instruções em um programa são executadas uma após a outra, na ordem em que são escritas. Isso é chamado de execução sequencial. No entanto, frequentemente precisamos "quebrar" essa sequência, seja por querer executar o mesmo comando (ou bloco de comandos) mais de uma vez ou por querer executar determinado conjunto de comandos somente se uma condição específica for verdadeira.

Em vista disso, existem as estruturas de controle, que podem ser os comandos de seleção ou de repetição (loops). Eles servem para alterarmos o fluxo normal de execução de nossos algoritmos e, assim, conseguirmos o objetivo. Nesta unidade veremos esses dois tipos de comando, começando pelos comandos de seleção e continuando com os comandos de repetição na linguagem de programação C.

Comandos de Seleção

Quando estamos desenvolvendo a lógica de nossos algoritmos, frequentemente precisamos realizar algum tipo de teste e, a partir desse resultado, executar um bloco de comandos ou não. Com este fim, temos os comandos de seleção, que podem ser divididos em comandos de seleção simples ou de seleção múltipla.

Os comandos de seleção que C possui são: *if* (seleção simples), *if-else* e switch (seleção múltipla).

Comando de seleção if

Usamos o comando *if* quando queremos testar uma condição e executar uma parte do código somente se ela for verdadeira. Este

comando possui a seguinte sintaxe:

if (condição) comandos;

lembrando que a declaração comandos pode ser um comando simples ou um comando composto. Se for um comando composto (mais de uma linha) devemos colocar delimitadores (‘{‘ e ‘}’) para indicar que mais de um comando deve ser englobado pelo if. Se for um comando simples, não há necessidade. Exemplificaremos isso logo abaixo (Figura 23.B):

Laboratorio.c

```
1 // Teste de maioridade
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void) {
6     printf("Digite sua idade: ");
7     fflush(stdout);
8     int idade;
9     scanf("%d", &idade);
10    if (idade >= 18)
11        printf("Você é maior de idade. Tem %d anos", idade);
12    return 0;
13 }
```

Figura 23 B. Exemplo de *if*

Veja a explicação linha por linha da aplicação da **Figura 23.B**:

Linha 1 – //Teste de maioridade: Comentário sobre a funcionalidade do programa;
Linhas 2 e 3 – #include<stdlib.h> e #include<stdio.h>: Inclusão das bibliotecas necessárias para a utilização das funções de entrada e saída;
Linha 5 – int main(void) {: Declaração da função principal e abertura do programa;
Linha 6 – printf(“Digite sua idade: ”);: Escrita de mensagem para o usuário;
Linha 7 – fflush(stdout);: Força a escrita imediata da mensagem anterior;
Linha 8 – int idade;: Declaração da variável que irá receber o valor da idade;
Linha 9 – scanf(“%d”, &idade);: Leitura do valor de idade a partir do teclado;
Linha 10 – *if* (idade >= 18): Comando de seleção usando a comparação da variável idade com o valor 18;

Linha 11 – printf(“Você é maior de idade. Tem %d anos.”, idade);: Escrita de mensagem. Esse comando só será executado se o valor de idade for maior que 18;
Linha 12 – return 0: Valor de retorno da função principal (main);
Linha 13 – }: Fechamento do programa.

Percebam que, nesse exemplo, usamos um dos operadores relacionais mostrados no **capítulo 1**. Dessa forma, o resultado da avaliação dessa expressão só pode ser verdadeiro ou falso, que são os valores que o comando if irá avaliar. Se o resultado da expressão for verdadeiro ele entra nos comandos do *if* e executa o printf da **linha 11**. Se o resultado da comparação idade >= 18 for falsa, então ele pulará para o próximo comando, logo após o if, na **linha 12**.

Nas figuras a seguir (Figura 24 e Figura 25) vemos o resultado da execução desse programa para cada um dos casos: idade >= 18 e idade < 18.

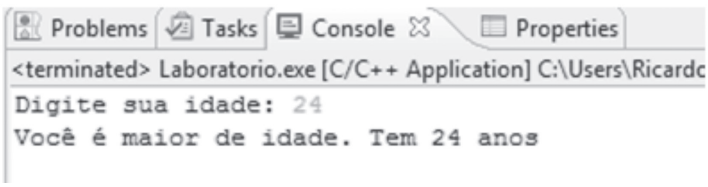


Figura 24. Resultado da execução com idade = 24

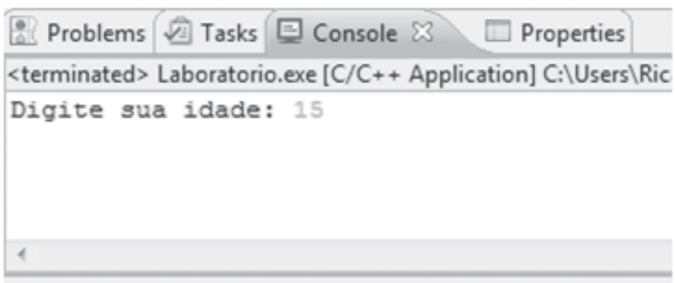


Figura 25. Resultado da execução com idade = 15

Vejamos agora outro exemplo usando um comando composto dentro do if para percebermos a diferença:

Laboratorio.c

```c
// Teste de maioridade
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("Digite sua idade: ");
    fflush(stdout);
    int idade;
    scanf("%d", &idade);
    if (idade >= 18) {
        printf("Você é maior de idade.\n");
        printf("Tem %d anos", idade);
    }
    return 0;
}
```

Figura 26 A. Segundo exemplo de *if*

Nesse caso, temos duas linhas de código a serem executadas se a condição idade > 18 for satisfeita. Com isso é necessário que coloquemos os delimitadores '{' (abre-chaves) e '}' (fecha-chaves), pois assim o programa irá reconhecer que os dois comandos estão englobados pela estrutura de seleção if. As figuras abaixo (Figura 26.B e Figura 27) mostram o resultado da execução desse programa para idade = 24 e idade = 13.

OBS.: A partir desse exemplo não mostraremos mais a explicação linha por linha. Isto por considerar que você já detém o conhecimento da estrutura básica dos programas em C, as quais sempre vinham sendo explicadas em todos os exemplos. Quaisquer dúvidas com essa estrutura, volte aos exemplos anteriores e verifique a explicação. De agora em diante serão explicadas apenas as linhas com conteúdo novo.

Problems | Tasks | Console | Properties

```
<terminated> Laboratorio.exe [C/C++ Application] C:\Users\Ricardo\
Digite sua idade: 24
Você é maior de idade.
Tem 24 anos
```

Figura 26 B. Resultado da execução com idade = 24

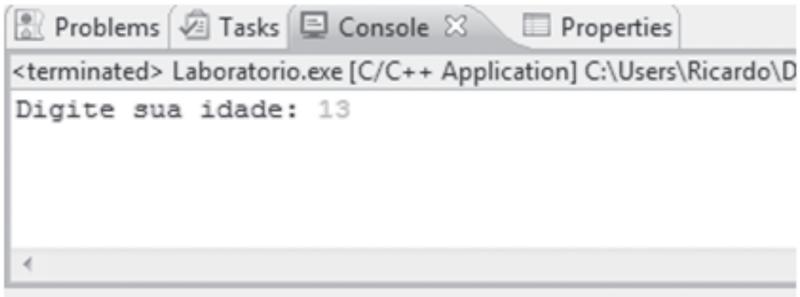


Figura 27. Resultado da execução com idade = 13

Neste terceiro exemplo o programa pedirá ao usuário para entrar com a média de um aluno qualquer e, dependendo do valor da média irá mostrar a mensagem “Aprovado”, se a média for maior ou igual a 7, “Prova final”, se a média for maior ou igual a 4 e menor que 7 e “Reprovado”, se a média for menor que 4.

Laboratorio.c

```
1 // Teste de aprovação
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void) {
6     printf("Digite sua média: ");
7     fflush(stdout);
8     float media;
9     scanf("%f", &media);
10    if (media >= 7) {
11        printf("Aprovado");
12    }
13    if (media < 7 && media >= 4) {
14        printf("Prova final");
15    }
16    if (media < 4) {
17        printf("Reprovado");
18    }
19    return 0;
20 }
```

Figura 28. Terceiro exemplo de *if*

Aqui, como tínhamos três casos diferentes de média para testar, fizemos três if, uma para cada caso. Percebam que na linha 13 usamos o operador booleano E (&&) para unir duas condições (media < 7 && media >= 4). A construção 4 <= media < 7, apesar de correta matematicamente, não está correta em nossa linguagem de programação. O modo correto de expressar é como na Figura 28.

Outro ponto a ser ressaltado nesse exemplo é que podemos usar no if, mesmo que o comando correspondente não seja composto, os

delimitadores '{' e '}'. Isso melhora a legibilidade do código. Abaixo temos o resultado da execução desse programa para os três casos possíveis:

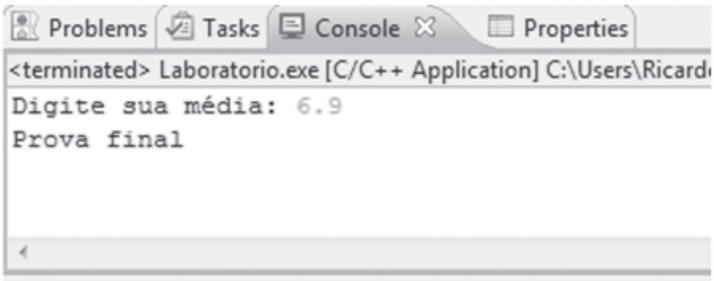


Figura 29. Resultado da execução para 4 <= media < 7

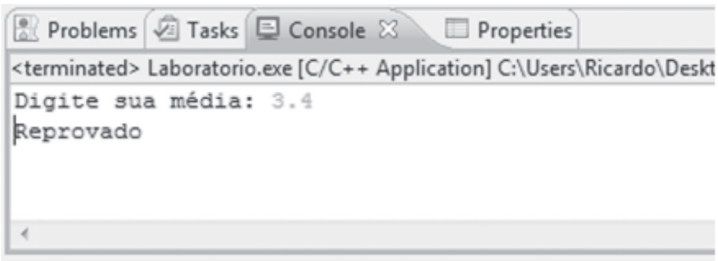


Figura 30. Resultado da execução para media < 4

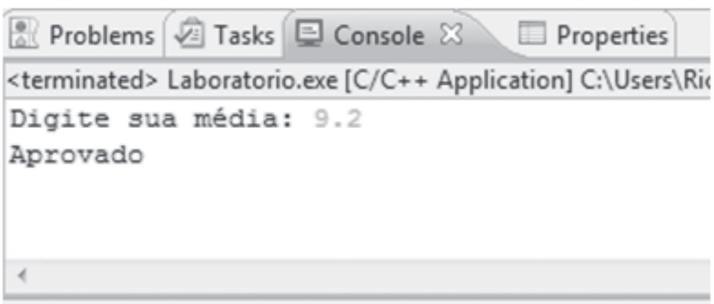


Figura 31. Resultado da execução para media > 7



Exercícios Propostos

1. (Resolvido) Faça um programa que leia três valores do teclado e verifique se o primeiro é maior que a soma dos outros dois.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>

int main() {
    int pri, seg, ter;
    printf("Digite o primeiro número: ");
    fflush(stdout);
    scanf("%d", &pri);
    printf("Digite o segundo número: ");
    fflush(stdout);
    scanf("%d", &seg);
    printf("Digite o terceiro número: ");
    fflush(stdout);
    scanf("%d", &ter);
    if (pri > seg + ter) {
        printf("Primeiro é maior que a soma dos dois");
    }
    if (pri <= seg + ter) {
        printf("Primeiro é menor que a soma dos dois");
    }
    return 0;
}
```

Execução:

Console

```
<terminated> Laboratorio.exe [C/C++ Application] D:\cp
Digite o primeiro número: 10
Digite o segundo número: 4
Digite o terceiro número: 3
Primeiro é maior que a soma dos dois
```

2. Faça um programa que leia um nome e informe se ele é igual ao seu dia de nascimento. Imprimir “NÚMERO CORRETO” se for igual e imprimir “NÚMERO INCORRETO”, caso contrário.

3. Faça um programa que receba as 3 notas de um aluno e calcule a média final, mostrando ainda o resultado, seguindo a seguinte regra: se a média for maior ou igual a 7, APROVADO; se a média for maior ou igual a 4 e menor que 7, PROVA FINAL; se a média for menor que 4, REPROVADO.

4. Faça um programa que leia a velocidade máxima permitida em uma avenida e a velocidade que um motorista estava dirigindo nela e calcule a multa que ele terá que pagar, sabendo que são pagos: R$50 se o motorista ultrapassar até 10km/h a velocidade permitida (ex: se a velocidade máxima permitida for 50km/h e ele estiver a 53 ou 60km/h); R$100 se o motorista ultrapassar de 11 a 30hm/h a velocidade permitida; e R$ 200 se ele estiver acima de 30km/h da velocidade permitida.

5. Faça um programa que leia dois valores reais (float) do teclado e calcule a divisão entre eles. Se o segundo for zero imprima (DIVISÃO POR ZERO)

6. Faça um programa que receba três números e imprima o menor deles.
7. Faça um programa que receba dois números e os imprima em ordem crescente.
8. Faça um programa que leia um número e verifique se ele é par ou ímpar.
9. Faça um programa que leia quatro números e imprima a soma dos que forem par.
10. Faça um programa que receba um ano e verifique se ele é bissexto. Um ano é bissexto se for divisível por quatrocentos ou se ele for divisível por quatro mas não por cem.

Comando de seleção dupla *if-else*

Em nosso terceiro exemplo de if vimos que testamos várias condições para a mesma variável, onde uma condição excluía a outra. Seria interessante usar um comando em que pudéssemos executar algo não somente para o valor verdadeiro de uma condição, mas também para o valor falso. Este é o objetivo do comando if-else, em que a parte do if funciona igual vimos anteriormente, mas, colocaremos um novo comando, o else, que indicará ao programa o que fazer se a condição testada no if for falsa. A sintaxe do comando é a seguinte:

if (condição) comandos_1; else comandos_2;

Da mesma forma que no if, os comandos podem ser simples ou compostos e usaremos delimitadores para separá-los. Vamos refazer o exemplo anterior usando, agora, esta estrutura (Figura 32).

Laboratorio.c

```
// Teste de aprovação
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("Digite sua média: ");
    fflush(stdout);
    float media;
    scanf("%f", &media);
    if (media >= 7) {
        printf("Aprovado");
    } else {
        if (media >= 4) {
            printf("Prova final");
        } else {
            printf("Reprovado");
        }
    }
    return 0;
}
```

Figura 32. Exemplo de *if-else*

Podemos perceber que o algoritmo ficou mais simples e representativo. O primeiro teste ficou igual, apenas verificando se a média é maior ou igual a sete. Se for, apenas escreve “Aprovado” e a execução do programa passa para a próxima linha depois da estrutura if-else que corresponde à **linha 19**.

Se esta condição não for verdadeira, ou seja, se a média for menor do que sete, o programa entra no bloco do else e irá verificar se a média é maior ou igual a quatro. Se for escreve “Prova final” e passa para a instrução logo após o if-else interno, **linha 18**. Se essa condição não for verdadeira, a execução irá entrar no else do *if-else* interno, irá mostrar “Reprovado” e continuará a execução também logo após o if-else interno, na **linha 18**.

As figuras a seguir mostram o resultado da execução desse programa para cada um dos três casos analisados:

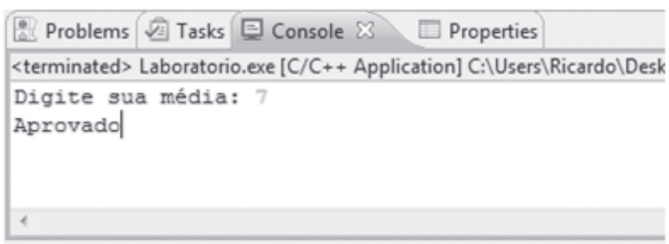


Figura 33. Resultado da execução para média > 7

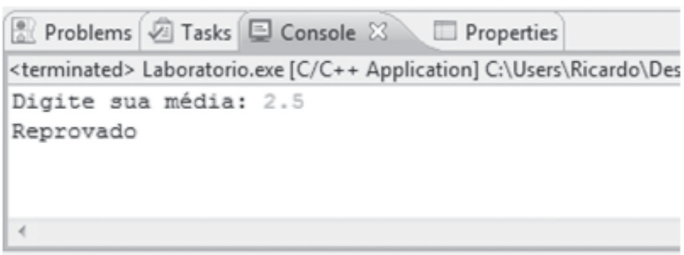


Figura 34. Resultado da execução para média < 4

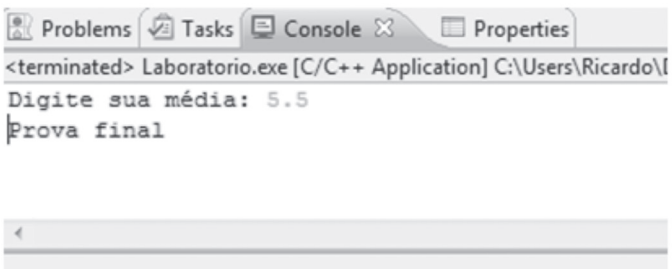


Figura 35. Resultado da execução para 4 <= média < 7

Esse nosso teste teve apenas três casos diferentes, mas no caso de existirem mais condições a serem testadas, nosso código tende a ficar cada vez mais bagunçado. Vejamos o exemplo a seguir:

Laboratorio.c

```
1// Teste de aprovação
2#include <stdio.h>
3#include <stdlib.h>
4
5int main(void) {
6    printf("Digite sua média: ");
7    fflush(stdout);
8    float media;
9    scanf("%f", &media);
10   if (media == 10.0) {
11       printf("100 por cento");
12   } else {
13       if (media >= 9) {
14           printf("De 90 a 99 por cento");
15       } else {
16           if (media >= 8) {
17               printf("De 80 a 89 por cento");
18           } else {
19               if (media >= 7) {
20                   printf("De 70 a 79 por cento");
21               } else {
22                   printf("Menos de 70 por cento");
23               }
24           }
25       }
26   }
27   return 0;
28}
```

Figura 36. Vários if-else aninhados

Colocar várias estruturas de controle uma dentro da outra se chama “aninhar” os comandos. Para evitar esta forma de escrita e melhorar a legibilidade de nosso código, a linguagem C nos propicia a forma de escrita compacta de cláusulas else juntamente com a próxima cláusula *if*, como demonstrado na **Figura 37**.

Laboratorio.c

```
1// Teste de aprovação
2#include <stdio.h>
3#include <stdlib.h>
4
5int main(void) {
6    printf("Digite sua média: ");
7    fflush(stdout);
8    float media;
9    scanf("%f", &media);
10   if (media == 10.0) {
11       printf("100 por cento");
12   } else if (media >= 9) {
13       printf("De 90 a 99 por cento");
14   } else if (media >= 8) {
15       printf("De 80 a 89 por cento");
16   } else if (media >= 7) {
17       printf("De 70 a 79 por cento");
18   } else {
19       printf("Menos de 70 por cento");
20   }
21   return 0;
22}
```

Figura 37. *if-else-if* compacto

Assim, as diferenças desse exemplo para o anterior são nas linhas 12, 14 e 16, onde juntamos a cláusula else com o próximo *if*, dando ao nosso código mais expressividade e legibilidade. Nota-se ainda que, usando essa estrutura, é necessário finalizarmos com uma última cláusula else (sem if), que casará quando todas as condições anteriores forem falsas, assim como está exemplificado na **linha 18**.

Se alguma das condições for verdadeira, será executado o bloco de comandos correspondente àquela cláusula e depois o controle será passado para fora de toda a estrutura *if-else-if*, na **linha 21**. Por exemplo, se as condições das **linhas 10 e 12** forem falsas e a da linha 14 for verdadeira, será executado o comando prinft da **linha 15** e o próximo comando a ser executado será o da **linha 21**. O resultado dessa execução está na **Figura 38**.

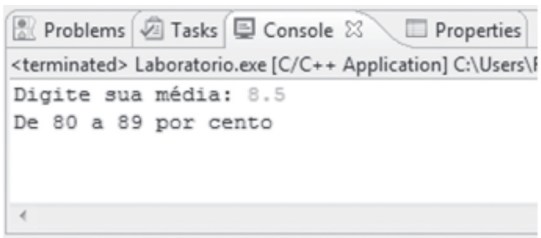


Figura 38. Exemplo de execução do exemplo de if-else-if

Exercícios Propostos

1. Faça um programa que leia os três coeficientes de uma equação e imprima as soluções, quando existirem. Lembrando que a existência das soluções depende de delta: a) Se delta for menos que zero não há soluções reais (imprima a mensagem “NÃO HÁ SOLUÇÕES REAIS”); b) Se delta for igual a zero existe apenas uma solução real (imprima a mensagem “EXISTE UMA SOLUÇÃO REAL, x = valor); c) Se delta for maior do que zero, temos duas soluções reais (imprima a mensagem “EXISTEM DUAS SOLUÇÕES REAIS, x1 = valor_1 e x2 = valor_2”).
2. Faça um programa que receba um mês do ano (em formato numérico) e imprima a quantidade de dias que ele possui (considere fevereiro contendo 28 dias).
3. (Resolvido) Faça um programa que receba três valores e imprima a soma dos dois maiores.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>

int main() {
    int pri, seg, ter;
    printf("Digite o primeiro número: ");
    fflush(stdout);
    scanf("%d", &pri);
    printf("Digite o segundo número: ");
    fflush(stdout);
    scanf("%d", &seg);
    printf("Digite o terceiro número: ");
    fflush(stdout);
    scanf("%d", &ter);
    if (pri < seg && pri < ter) {
        printf("Resultado: %d", seg + ter);
    } else if (seg < ter) {
        printf("Resultado: %d", pri + ter);
    } else {
        printf("Resultado: %d", pri + seg);
    }
    return 0;
}
```

Execução:

Console

<terminated> Laboratorio.exe [C/C++ Applic

```
Digite o primeiro número: 10
Digite o segundo número: 4
Digite o terceiro número: 12
Resultado: 22
```

4. Faça um programa que leia o número de lados de um polígono regular, e a medida do lado. Calcular e imprimir o seguinte:

a. Se o número de lados for igual a três, escrever TRIÂGULO e o valor do seu perímetro.

b. Se o número de lados for igual a quatro, escrever QUADRADO e o valor da sua área.

c. Se o número de lados for igual a cinco escrever PENTÁGONO.

5. Em qualquer outra situação escrever Polígono não identificado.

6. Faça um programa que leia três valores e verifique se formam um triângulo (o valor de cada lado tem que ser a soma dos outros dois). Caso positivo, classifique esse triângulo como triângulo equilátero (possui os três lados iguais), triângulo isósceles (possui dois lados iguais) e triângulo escaleno (possui três lados diferentes).

7. Faça um programa que leia o valor de três ângulos de um triângulo e escreva se o triângulo é acutângulo (Possui 3 ângulos agudos – menores que 90 graus), retângulo (Possui um ângulo reto – 90 graus), obtusângulo

(possui um ângulo obtuso – maior que 90 graus).
8. Faça um programa que leia um número e imprima se ele é par ou ímpar.
9. Utilize a estrutura “if” para fazer um programa que retorne o nome de um produto a partir do código do mesmo. Considere os seguintes códigos:

a. 001: Parafuso;
b. 002: Porca;
c. 003: Prego;
d. Para qualquer outro código indicar: diversos!

10. Escreva um algoritmo que solicita ao usuário para digitar um número inteiro positivo, e mostre-o por extenso. Este número deverá variar entre 1 e 10. Se o usuário introduzir um número que não pertença a este intervalo, mostre a frase “número inválido”.
11. Faça um programa que leia dois números (x e y) e verifique se o primeiro é divisível pelo segundo. Se o primeiro for divisível pelo segundo, imprima: “’x’ é divisível por ‘y’”. Se o primeiro não for divisível pelo segundo, imprima: “’x’ não é divisível por ‘y’”.

O comando *switch*

Em nosso último exemplo de *if-else-if*, pudemos verificar a existência de várias condições consecutivas testando a mesma variável. Em casos como esses, o comando switch é mais recomendável, pois, sendo um comando de seleção múltipla, dá mais expressividade e legibilidade para estruturas onde testamos inúmeros casos para a mesma variável. Temos, a seguir, a sintaxe para este comando:

```
switch (variável) {
        case contante_1: comandos_1;
                break;
        case contante_2: comandos_2;
                break;
        ...
        default: comandos_default;
                break;
    }
```

Com este comando, passamos a variável que queremos testar e, dependendo do seu valor, o programa irá executar um caso diferente. Apesar do poder dessa estrutura, temos algumas limitações, como, por exemplo: não se podem passar expressões para serem testadas, apenas variáveis simples. Os tipos de variáveis que podem ser testadas são somente int e char, não sendo possível passar *float, double ou string*, por exemplo.

Cada caso é indicado pela palavra-chave case, seguida do valor de teste e de dois pontos (:). Após os dois pontos serão colocados os comandos correspondentes a cada caso. Ao final dessa sequência de comandos, é necessário que coloquemos o comando *break*, pois ele "quebrará" a execução e redirecionará o fluxo para fora do *switch*. Sendo assim, é o *break* que informa até onde vão os comandos de cada opção, não sendo necessários delimitadores para cada caso (apenas para o próprio comando *switch*).

Ao final de todos os casos, podemos especificar um caso *default*, que funcionará da mesma forma que um else em uma sequência de *if-else-if*. Ou seja, se nenhum dos casos da estrutura *switch* for casar com o valor da variável de entrada, o programa entrará no caso *default* e executará as instruções ali especificadas. Para esse caso, não é necessário colocarmos o comando *break* finalizando o mesmo, visto que ele será sempre o último caso, no entanto, para melhorar a legibilidade de nossos programas, o ideal é não nos esquecermos de colocá-lo, mesmo nesse caso.

Vejamos o exemplo (Figura 39) para entendermos o funcionamento desta estrutura:

Laboratorio.c

```
1// Escolha de veículo e retorno de pedágio correspondente
2#include <stdio.h>
3#include <stdlib.h>
4
5int main(void) {
6    int tipoVeiculo;
7    double pedagio;
8    printf("Informe o tipo de veículo: ");
9    fflush(stdout);
10   scanf("%d", &tipoVeiculo);
11   switch(tipoVeiculo) {
12       case 1: printf("Carro de passeio.\n");
13               pedagio = 0.50;
14               break;
15       case 2: printf("Ônibus.\n");
16               pedagio = 1.00;
17               break;
18       case 3: printf("Caminhão.\n");
19               pedagio = 2.00;
20               break;
21       default: printf("Veículo desconhecido.\n");
22               pedagio = 0.00;
23               break;
24   }
25   printf("Valor do seu pedágio: R$ %.2f", pedagio);
26   return 0;
27}
```

Figura 39. Exemplo de switch

Nesse exemplo pedimos para o usuário entrar com um número identificando o tipo de veículo. Tipo 1 corresponde a “carro de passeio” (linha 12), nesse caso o pedágio será de R$ 0,50 (linha 13). O tipo 2 corresponde a “ônibus” (linha 15), para eles o pedágio será de R$ 1,00 (linha 16). No último caso, tipo 3, temos “caminhão” (linha 18), com pedágio de R$ 2,00 (linha 19). Se nenhum desses valores for recebido, o sistema retornará a mensagem “veículo desconhecido” (linha 21).

Ao final, linha 24, imprimimos a mensagem “valor do seu pedágio: R$” e o valor selecionado logo em seguida. Atentem para a formatação do valor para saída na linha 24 (%.2f), nesse caso estamos indicando que será impresso um valor real (f) com duas casas decimais (.2). Abaixo temos as saídas do programa para cada uma das opções escolhidas (Figura 40, 41, 42 e 43).

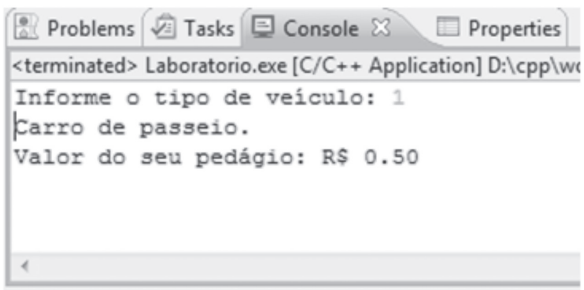


Figura 40. Resultado da execução para tipo de veículo 1

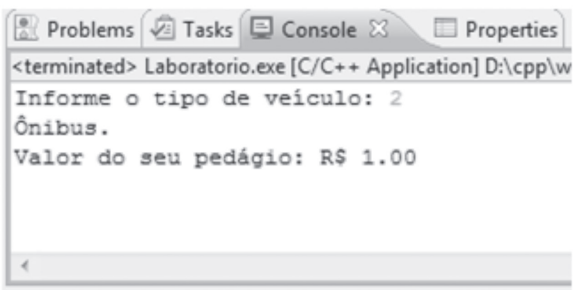


Figura 41. Resultado da execução para tipo de veículo 2

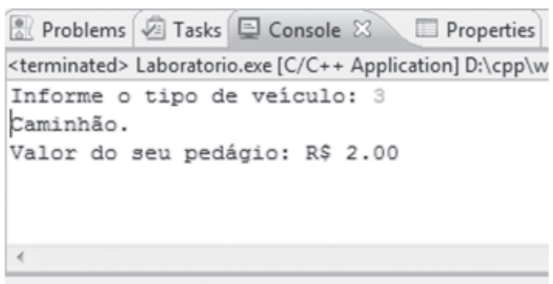


Figura 42. Resultado da execução para tipo de veículo 3

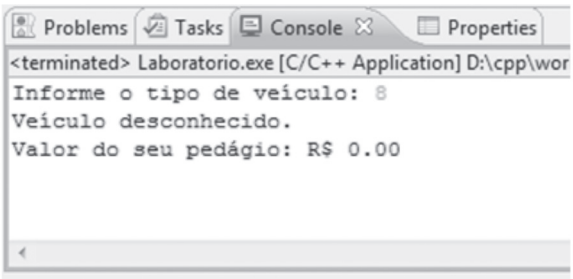


Figura 43. Resultado da execução para tipo de veículo 8

Podemos ter ainda 2 (ou mais) case para a mesma seção de código. Para isto, basta não colocarmos o comando break entre as opções como no exemplo a seguir:

```
case 'a':
case 'A':
        printf("Você escolheu a letra A.\n");
        break;
```

Figura 44. Exemplo de 2 cases para o mesmo conjunto de comandos

Dessa forma, se a opção escolhida pelo usuário for a letra ‘a’ minúscula ou a letra ‘A’ maiúscula, os comandos executados são o mesmo, ou seja, mostrará a mensagem “Você escolheu a letra A”, para o usuário.



Exercícios Propostos

1. Faça um programa que exiba um menu ao usuário com as seguintes opções: 1-Inclusão, 2-Exclusão, 3-Sair. Ao usuário selecionar uma opção o programa deverá mostrar um texto correspondendo à opção selecionada. Mostre “opção inválida”, caso seja diferente de 1, 2 ou 3.

2. (Resolvido) Faça um programa que receba o valor inteiro e retorne o mês do ano equivalente (1 = janeiro, 2 = fevereiro, 3 = março, ..., 12 = dezembro). Se o valor recebido for maior que 12 ou menor que 1 deverá

imprimir “mês inválido”.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>

int main() {
    int mes;
    printf("Digite o número do mês: ");
    fflush(stdout);
    scanf("%d", &mes);
    switch (mes) {
        case 1: printf("Janeiro"); break;
        case 2: printf("Fevereiro"); break;
        case 3: printf("Março"); break;
        case 4: printf("Abril"); break;
        case 5: printf("Maio"); break;
        case 6: printf("Junho"); break;
        case 7: printf("Julho"); break;
        case 8: printf("Agosto"); break;
        case 9: printf("Setembro"); break;
        case 10: printf("Outubro"); break;
        case 11: printf("Novembro"); break;
        case 12: printf("Dezembro"); break;
        default: printf("Mês inválido!"); break;
    }
    return 0;
}
```

Execução:

Console

```
<terminated> Laboratorio.exe [C/C++ Ap
Digite o número do mês: 3
Março
```

3. Faça um programa que receba dois valores e exiba um menu para o usuário escolher uma das quatro operações aritméticas. Baseado nessa escolha o programa deverá calcular e imprimir o resultado.
4. Faça um programa que leia um número e o escreva por extenso se estiver no intervalo de 0 a 20. Caso contrário, escreva “número inválido”.
5. Faça um programa que receba a letra inicial do estado civil de uma pessoa e imprima a descrição: Casado(a), Solteiro(a), Viúvo(a), desquitado(a), Divorciado(a), Inválido.
6. Faça um programa que leia o preço e a categoria de um produto. O programa deve calcular e mostrar o reajuste de acordo com a categoria: A = 50%, B = 25%, C = 15% e outros = 5%.
7. Faça um programa que leia uma letra do alfabeto e imprima das mensagens a seguir a correspondente para cada caso: “Vogal maiúscula”, “Vogal minúscula”, “Consoante”.
8. Faça um programa que leia a categoria da carteira de motorista do usuário e imprima os tipos de veículos que ele pode dirigir.

9. Utilize a estrutura *switch* para fazer um programa que retorne o nome de um produto a partir do seu código. Considere os seguintes códigos:

a. 001: Parafuso;
b. 002: Porca;
c. 003: Prego;
d. Para qualquer outro código indicar: diversos!

10. Faça um programa que receba um código equivalente às 27 unidades federativas brasileiras e imprima o nome e a capital correspondente. Receba números de 1 a 27, considerando os estados organizados em ordem alfabética.

Comandos de Repetição

Em nossos programas, frequentemente necessitamos que uma parte do nosso código seja repetida uma determinada quantidade de vezes. Para isso, a linguagem de programação C disponibiliza três operadores de repetição, são eles: *for, while e do-while*. A seguir, detalharemos cada um deles.

O comando for

O primeiro comando de repetição a ser apresentado será o *for*. Especialmente adequado quando conhecemos a quantidade de iterações que nosso bloco de comandos terá, o comando *for* necessita de três parâmetros: a inicialização, a condição para continuidade e o incremento. A sintaxe desse comando é a seguinte:

for (inicialização; condição; incremento) comando;

O primeiro parâmetro indica o início da contagem. Através dele atribuímos o valor inicial da variável que controlará o laço. O segundo parâmetro representa a condição de continuidade, ou melhor, a condição que fará o laço continuar a repetir. Sendo assim, enquanto essa condição for verdadeira a repetição irá continuar. Por último, temos o incremento. Esse parâmetro indicará a variação da variável de controle em cada passo da repetição.

Sendo assim, a estrutura *for* irá inicializar a variável de contagem e executar o comando (simples ou composto) passado para ele. A cada passo irá executar o incremento e testar novamente a condição para continuar ou não a repetição antes de executar novamente o comando.

Quando a condição for falsa, a repetição terminará e o programa passará para a próxima instrução, logo após o *for*. Veremos o funcionamento na prática no exemplo a seguir (Figura 45):

Laboratorio.c

```
1 // Escrever os numeros de 1 a 10
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void) {
6     int i;
7     for (i = 1; i <= 10; i++) {
8         printf("%d ", i);
9     }
10    return 0;
11 }
```

Figura 45. Utilização do *for*

Nesse exemplo temos, na **linha 7**, um comando *for*. Na linha anterior (linha 6) declaramos uma variável inteira i que vai controlar a quantidade de iterações do *loop*. Na declaração do *for* inicializamos o contador com valor 1 (i = 1). Como condição de parada temos a condição i <= 10, dessa forma, a repetição continuará enquanto o valor de i não ultrapassar o valor 10. Por último temos o incremento (i++), que significa que a variável i aumentará de 1 em 1 a cada iteração (o operador i++ equivale a i+=1 que, por sua vez, equivale a i = i + 1).

Assim, o comando da **linha 8** será repetido 10 vezes. Dessa forma, a cada passo, o comando printf escreverá o valor da variável i seguido de um espaço em branco. Com isso, a saída desse programa é a seguinte (Figura 46):

Problems | Tasks | Console | Properties

```
<terminated> Laboratorio.exe [C/C++ Application] D:\cpp\w
1 2 3 4 5 6 7 8 9 10
```

Figura 46. Resultado da execução

A seguir (Figura 47) temos outro exemplo de utilização do comando *for*. Nesse caso, queremos imprimir os números ímpares de 1 a 50. Como os números ímpares variam de 2 em 2, usamos outro comando para

incremento. Vejamos o exemplo:

Laboratorio.c

```
1 // Escrever os numeros ímpares de 1 a 50
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void) {
6     int i;
7     for (i = 1; i <= 50; i += 2) {
8         printf("%d ", i);
9     }
10    return 0;
11 }
```

Figura 47. Imprimir os números ímpares de 1 a 50

Nesse exemplo a única diferença está no parâmetro de incremento. Aqui usamos i += 2, que indica que, a cada passo, o valor da variável i mudará de 2 em 2, pois é o que necessitamos para imprimir os números ímpares. O resultado da execução deste programa está na Figura 48, a seguir:

Problems | Tasks | Console | Properties

```
<terminated> Laboratorio.exe [C/C++ Application] D:\cpp\workspace\Laboratorio\Debug\Laboratorio.exe
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49
```

Figura 48. Resultado da execução: impressão dos números ímpares de 1 a 50

Exercícios Propostos

2. Faça um programa que calcule a soma dos 100 primeiros números naturais.
3. Faça um programa que imprima os múltiplos de 9 menores que 300.
4. Faça um programa que calcule e imprima os números divisíveis por 3 entre 10 e 450.
5. (Resolvido) Faça um programa que calcule os divisores de um número qualquer, informado pelo usuário.
6. Faça um programa que receba as idades dos 30 alunos de uma turma e, ao final, calcule e escreva a média das idades desses alunos.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>

int main() {
    int n;
    printf("Digite o número: ");
    fflush(stdout);
    scanf("%d", &n);
    int i;
    printf("Divisores de %d: ", n);
    for (i = 1; i <= n; i++) {
        if (n % i == 0) {
            printf("%d, ", i);
        }
    }
    return 0;
}
```

Execução:

```
Console
<terminated> Laboratorio.exe [C/C++ Application] D:\cp
Digite o número: 18
Divisores de 18: 1, 2, 3, 6, 9, 18,
```

7. Faça um programa que leia 10 valores, mostre quantos deles estão no intervalo de [10, 100] e quantos deles não estão.

8. Faça um programa que calcule o fatorial de um número qualquer (Lembrando o cálculo do fatorial: n! = n * n-1 * n-2 * ... * 2 * 1. Por exemplo: 5! = 5 * 4 * 3 * 2 * 1 = 120).

9. Faça um programa que imprima a tabuada do número 4, com as quatro operações, no seguinte formato:

0 + 4 = 4	4 - 4 = 4	0 * 4 = 0	0 / 4 = 0
1 + 4 = 5	5 - 4 = 5	1 * 4 = 4	4 / 4 = 1
2 + 4 = 6	6 - 4 = 6	2 * 4 = 8	8 / 4 = 2
3 + 4 = 7	7 - 4 = 7	3 * 4 = 12	12 / 4 = 3
4 + 4 = 8	8 - 4 = 8	4 * 4 = 16	16 / 4 = 4
5 + 4 = 9	9 - 4 = 9	5 * 4 = 20	20 / 4 = 5
6 + 4 = 10	10 - 4 = 10	6 * 4 = 24	24 / 4 = 6
7 + 4 = 11	11 - 4 = 11	7 * 4 = 28	26 / 4 = 7
8 + 4 = 12	12 - 4 = 12	8 * 4 = 32	32 / 4 = 8
9 + 4 = 13	13 - 4 = 13	9 * 4 = 36	36 / 4 = 9
10 + 4 = 14	14 - 4 = 14	10 * 4 = 40	40 / 4 = 10

10. Faça um programa que imprima a tabuada de um número qualquer. Similar à questão anterior, no entanto, o usuário indicará a tabuada de qual número o programa irá mostrar.

11. Faça um programa que receba um número e verifique se ele é primo ou não.

O comando while

O comando *while* também serve para repetirmos um comando (simples ou composto) determinada quantidade de vezes. Diferentemente do *for*, este comando é mais adequado quando não sabemos, a princípio, a quantidade de repetições que irá ocorrer. Isso acontece porque não há contador, apenas um teste de continuidade para verificar se a repetição irá parar ou não. Vejamos a sintaxe deste comando:

while (condição) comando;

Aqui teremos apenas uma condição controlando a repetição. Enquanto essa condição for verdadeira a repetição continuará. Quando a condição não for mais verdadeira, a repetição terminará. Dessa forma, é importante que, em algum momento, forcemos essa condição a ser falsa, pois, caso contrário, o programa nunca sairá da repetição. Esse é o erro chamado de *loop* infinito, que trava o programa dentro do *while* e não mais sai de lá.

Vejamos um exemplo de utilização do *while* a seguir (Figura 49). Nesse exemplo, iremos calcular a soma de todos os números digitados pelo usuário. Nosso valor para saída da repetição será -1, ou seja, quando o usuário entrar com -1 como valor o programa sairá da repetição.

Laboratorio.c

```
1 // Soma dos números digitados pelo usuário
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void) {
6     float numero;
7     float soma = 0;
8     while (numero != -1) {
9         printf("Digite um número (-1 para sair): ");
10        fflush(stdout);
11        scanf("%f", &numero);
12        if (numero != -1) {
13            soma = soma + numero; // ou soma += numero
14        }
15    }
16    printf("O resultado da soma é: %f:", soma);
17    return 0;
18 }
```

Figura 49. Exemplo de utilização do while

Neste programa, as linhas 9, 10 11, 12, 13 e 14 irão se repetir através do comando *while*. Essa repetição terminará no momento em que a variável número assumir o valor -1. Isso acontecerá quando, na linha 11, o usuário inserir o valor adequado. Enquanto não houver essa entrada, a repetição irá continuar.

É importante observar ainda que, quando a entrada for -1, esse valor não deve ser considerado para os cálculos, ou seja, para a soma dos números inseridos pelo usuário. Desta forma, precisamos colocar um comando if na linha 12 que só autorizará o cálculo a ser atualizado se o valor de numero for diferente de -1. Abaixo (Figura 50) temos um exemplo de saída para este programa:

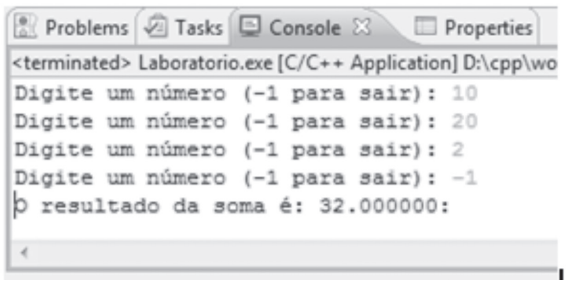


Figura 50. Execução do exemplo de while

Exercícios Propostos

1. Faça um programa que escreva os números de 10 a 500.
2. Faça um programa que crie um menu com as opções a seguir, sendo que, quando o usuário selecionar uma opção, o programa apenas irá apresentar o texto “Opção nome_da_opção escolhida”, com exceção da opção 4 que fará o programa terminar sua execução:

    1 – Cadastrar
    2 – Alterar
    3 – Excluir
    4 – Sair

3. Faça um programa que leia dois números inteiros, o primeiro é o valor inicial de um contador, e o segundo é o valor final do contador (o valor inicial fornecido é inferior ao valor final). Usando o comando *while*,

escreva na tela uma contagem que comece no primeiro número lido, escreva os números seguintes colocando apenas um número em cada nova linha da tela, até chegar ao valor final indicado. Exemplo de tela de saída:

Entre com o número inicial da contagem: 5
Entre com o número final da contagem: 9 5 6 7 8 9

4. Fazer um programa que calcule e imprima o fatorial de um número fornecido pelo usuário, usando o comando *while*. Repetir a execução do programa tantas vezes até o usuário responder não (Lembrando o cálculo do fatorial: n! = n * n-1 * n-2 * ... * 2 * 1. Por exemplo: 5! = 5 * 4 * 3 * 2 * 1 = 120). Exemplo de tela de saída:

Entre com um número: 5
O fatorial de 5 é 120
Outro número (sim/não)? não.

5. Ler o nome de um aluno e suas duas notas A e B, e após calcular a média ponderada entre estas notas (A tem peso 1 e B tem peso 2). Repetir este procedimento para uma turma composta por três alunos, usando o comando *while*.

6. Alterar o programa anterior de maneira que possamos trabalhar com turmas compostas por um número variável de alunos. Após calcular e imprimir a média de um aluno, exibir uma mensagem perguntando ao usuário se existem mais alunos (resposta: sim / não). Se tiver mais alunos, continuar o procedimento de leitura das notas e o cálculo da média até que o usuário responda “não”.

7. Alterar o programa anterior de maneira a validar as notas fornecidas pelo usuário (notas devem ser valores positivos entre 0.0 e 10.0). Indicar ao usuário se a nota fornecida é inválida e pedir para fornecer uma nova nota, repetindo este processo até que o usuário informe uma nota correta. Usar uma variável de controle booleana (flag) dentro do laço *while* de leitura da nota, e gerar uma saída conforme o exemplo de tela de saída abaixo.

Entre com o nome do aluno: João da Silva
Entre com a nota A: 15.3
ERRO: Nota inválida! Digite novamente a nota.
Entre com a nota A: 5.0
Entre com a nota B: 6.0
O aluno João da Silva tem uma média: 5.66
Continuar (sim/não)? não

8. Escrever um programa que calcule todos os números divisíveis por certo valor indicado pelo usuário (o resto da divisão por este número deve ser igual a zero), compreendidos em um intervalo também especificado pelo usuário. O usuário deve entrar com um primeiro valor correspondente ao divisor e após ele vai fornecer o valor inicial do intervalo seguido do valor final deste intervalo. Exemplo:

Entre com o valor do divisor: 3
Início do intervalo: 17
Final do intervalo: 29
Números divisíveis por 3 no intervalo de 17 a 29: 18 21 24 27.

9. Seu Manoel tem um comércio e precisa de um programa que lhe apresente o total de produtos vendidos e o valor total de vendas. Faça um programa que leia a quantidade de cada produto e seu valor e apresente esses resultados (Continuar inserindo valores até que o usuário digite a -1).

10. (Resolvido) Escreva um programa que imprima na tela a série de Fibonacci até um número dado. Esta série começa com dois números 1's e os próximos números são obtidos pela soma dos dois anteriores.
Ex: 1 1 2 3 5 8 13 21 34 55.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>

int main() {
    int n;
    printf("Digite o número: ");
    fflush(stdout);
    scanf("%d", &n);
    int proximo = 1, anterior = 1, atual = 1;
    printf("Fibonacci até %d: 1, 1", n);
    while (proximo < n) {
        printf(", %d", proximo);
        proximo = anterior + atual;
        anterior = atual;
        atual = proximo;
    }
    return 0;
}
```

Execução:

Console

```
<terminated> Laboratorio.exe [C/C++ Application] D:\cpp\workspace\Laboratorio\Debug\
Digite o número: 100
Fibonacci até 100: 1, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89
```

O comando do-while

O comando *do-while* é bastante parecido com o *while*, através dele podemos repetir um comando, ou sequência de comandos, várias vezes. A diferença maior reside no fato de o teste de continuidade no *while* ser logo no início do comando, enquanto que no *do-while* este teste é realizado ao final. Sendo assim, se a condição de continuidade não for verdadeira antes da entrada na repetição, o while não irá executar nenhuma vez. Entretanto, no *do-while*, pelo fato do teste ser depois, os comandos correspondentes serão executados ao menos uma vez. Vejamos a sintaxe desse comando:

```
do {
} while(condição);
```

Nesta estrutura de controle a repetição será controlada pela condição colocada depois do fechamento do bloco iniciado pelo do. Dessa forma, a condição só é testada depois da execução dos comandos contidos no bloco e, caso seja verdadeira, a repetição continuará, caso seja falsa, a repetição irá parar e o programa continuará sua execução logo após o comando do-while. Com isso, vemos que os comandos devem ser executados pelo menos uma vez, mesmo se a condição for falsa. Vejamos o exemplo a seguir (Figura 51):

Laboratorio.c

```
1  // Escrita dos valores da variável número
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void) {
6      int numero = 0;
7      do {
8          printf("%d ", numero);
9          numero--;
10     } while (numero > 0);
11     return 0;
12 }
```

Figura 51. Exemplo de utilização do do-while

Nesse exemplo iniciamos o número com 0. Na repetição vamos decrementando (operador --) o valor de número e mostrando os valores enquanto o valor da variável for maior que zero. Note que, nesse caso, a

saída será apenas o número 0, pois após o primeiro passo da repetição, o valor de número será -1 e a repetição terminará. Vejamos a saída desse programa na **Figura 52**:

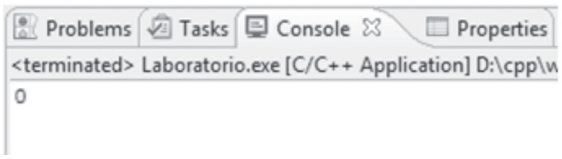


Figura 52. Resultado da execução do exemplo de do-while

Através desse exemplo podemos perceber a diferença de funcionamento do while para o do-while. Se tivéssemos usado o comando while, como mostra a Figura 53 a seguir, a repetição não seria executada, pois a condição seria falsa e o programa não executaria os comandos dentro do while.

Laboratorio.c

```
1 // Escrita dos valores da variável número
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void) {
6     int numero = 0;
7      while (numero > 0) {
8         printf("%d ", numero);
9         numero--;
10    }
11    return 0;
12 }
```

Figura 53. Implementação do exemplo de do-while com while

A diferença na execução desse programa pode ser vista na Figura 54. Vejamos que, usando o while, o resultado da execução é diferente:

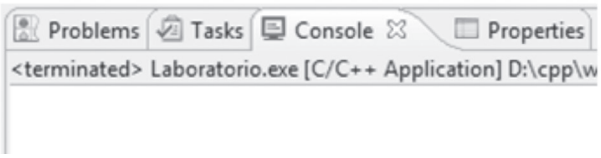


Figura 54. Resultado da execução: nenhuma saída

Podemos, ainda, usar esses dois comandos para implementar o mesmo algoritmo. Vejamos, na Figura 55, uma implementação do exemplo de while (Figura 49) usando o comando do-while. Nesse caso, o resultado da execução será exatamente o mesmo de quando foi usado a estrutura while.

Laboratorio.c

```
1 // Soma dos números digitados pelo usuário
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void) {
6     float numero;
7     float soma = 0;
8     do {
9         printf("Digite um número (-1 para sair): ");
10        fflush(stdout);
11        scanf("%f", &numero);
12        if (numero != -1) {
13            soma = soma + numero; // ou soma += numero
14        }
15    } while (numero != -1);
16    printf("O resultado da soma é: %f:", soma);
17    return 0;
18 }
```

Figura 55. Reimplementação do exemplo de while com do-while

A diferença na implementação é bem pequena. Na linha 8, ao invés de colocarmos o comando while, colocamos a estrutura do, que inicia o bloco de repetição para a estrutura do-while. E, ao final, na linha 15, ao invés de apenas fecharmos a chave, colocamos, após a chave, o comando while com o teste a ser executado para verificar se a repetição continuará ou não. Vemos, na Figura 56, a seguir, que o resultado da execução permanece o mesmo:

Problems | Tasks | Console | Properties

```
<terminated> Laboratorio.exe [C/C++ Application] D:\cpp\w
Digite um número (-1 para sair): 10
Digite um número (-1 para sair): 20
Digite um número (-1 para sair): 12
Digite um número (-1 para sair): -1
O resultado da soma é: 42.000000:
```

Figura 56. Resultado da execução do exemplo de do-while

Com isso, o que podemos perceber é que os comandos while e do-while podem ser intercambiáveis, dependendo da aplicação que estamos pretendendo fazer. Na verdade, poderíamos fazer este mesmo algoritmo também com a estrutura for (tente como exercício), produzindo a mesma saída. Sendo assim, qualquer programa que necessite de repetições pode ser feito usando qualquer das três estruturas de repetição vistas aqui.

No entanto, como vimos nesta unidade, cada comando tem os casos onde é mais indicado utilizá-lo. Portanto, devemos estar atentos a isso para que a sequência lógica de nossos algoritmos fique a mais simples possível (for para repetição número definido, while para repetição sem número definido e com teste antes e do-while para repetição sem número definido e com teste depois).

Exercícios Propostos



1. Escreva um programa que, dados 15 números, imprima seus quadrados.
2. Escreva um programa que, dados dois números, imprima o MMC.
3. Escreva um programa que, dados dois números, imprima o MDC.
4. Escreva um programa que imprima os seguintes dados na tela.

1-1 1-2 1-3 1-4 1-5
2-1 2-2 2-3 2-4 2-5
3-1 3-2 3-3 3-4 3-5
4-1 4-2 4-3 4-4 4-5
5-1 5-2 5-3 5-4 5-5

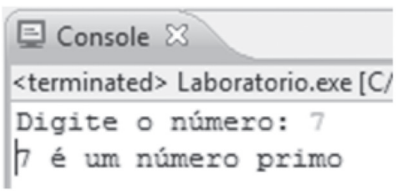
5. (Resolvido) Escreva um programa que, dada uma entrada, diga se o número é primo ou não.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>

int main() {
    int n;
    printf("Digite o número: ");
    fflush(stdout);
    scanf("%d", &n);
    int i = 1, div = 0;
    do {
        if (n % i == 0) {
            div++;
        }
        i++;
    } while (i <= n);
    if (div > 2) {
        printf("%d não é um número primo", n);
    } else {
        printf("%d é um número primo", n);
    }
    return 0;
}
```

Execução:



6. Escreva um programa que imprima os primos até um número digitado.
7. Escreva um programa que imprima os números de 100 a 600 que são múltiplos de 5.
8. Escreva um programa onde o usuário diz quantos números quer digitar, em seguida solicite a ele que digite todos os números e diga qual o maior número daqueles digitados.
9. A série de RICCI difere da série de FIBONACCI porque os dois números iniciais são dados pelo usuário e os outros termos são gerados a partir da soma dos dois anteriores. Escreva um programa que imprima os 20 primeiros números a partir de duas entradas do usuário.
10. Dado um país A, com 5.000.000 de habitantes e uma taxa de natalidade de 3% ao ano, e um país B com 7.000.000 de habitantes e uma taxa de natalidade de 2% ano. Calcular e imprimir o tempo necessário para que a população do país A ultrapasse a do país B.

Extra: comandos break e continue

É sabido que os comandos for, while e do-while servem para repetir determinada parte do código. No entanto, muitas vezes queremos alterar a sequência natural dessas repetições, ou quebrando a repetição ou ignorando determinados passos se uma condição específica for satisfeita. É nesse ponto que entram os comandos break e continue.

O comando break

Já vimos esse comando quando estudamos a estrutura de seleção múltipla switch. Aqui ele terá uma função bastante parecida. Dentro de uma repetição, se o programa alcançar um comando break a repetição irá parar naquele momento sem executar os passos faltantes. Ou seja, ele servirá para quebrar a repetição e continuar a execução do programa fora do laço. Vejamos o exemplo a seguir (Figura 57).

Laboratorio.c

```
1  // Soma dos números digitados pelo usuário
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void) {
6      float numero;
7      float soma = 0;
8      while (1) {
9          printf("Digite um número: ");
10         fflush(stdout);
11         scanf("%f", &numero);
12         if (numero == -1) {
13             break;
14         }
15         soma = soma + numero; // ou soma += numero
16     }
17     printf("O resultado da soma é: %f:", soma);
18     return 0;
19 }
```

Figura 57. Exemplo de utilização do break

Nesse exemplo, que aparentemente teria um loop infinito por causa da condição do while ser 1 (1 e qualquer coisa diferente de zero é verdadeiro em C, 0 é falso), isso não acontecerá. A saída da repetição se dará da mesma forma que os exemplos anteriores, através da entrada do número -1. Mas, para esse programa, quando isso acontecer, o if da linha 12 será verdadeiro e o programa irá para a linha 13 e executará o break, saindo da repetição. Temos, aqui, mais uma forma de implementar o mesmo algoritmo visto anteriormente.

O comando continue

A instrução continue possui a função de saltar, mas apenas o passo corrente, de um comando iterativo. Ou seja, durante uma repetição, se o programa alcançar o comando continue, aquele passo da repetição será abortado e o programa continuará no início da próxima iteração. Vejamos o exemplo a seguir (Figura 58):

Laboratorio.c

```
1  // Escrever os números de 1 a 10
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void) {
6      int i;
7      for (i = 1; i <= 10; i++) {
8          if (i == 5) {
9              continue;
10         }
11         printf("%d ", i);
12     }
13     return 0;
14 }
```

Figura 58. Exemplo de continue

Analisando este exemplo, temos um comando de repetição for (linha 7) controlado pela variável i que inicia em 1 e vai até 10. No entanto, quando o valor de i for 5, o teste do if na **linha 8** resultará verdadeiro e o continue da linha 9 será executado. Nesse caso, a repetição passará para o próximo passo, com i = 6, e o restante do passo anterior não será executado. Portanto, a saída exibida por esse programa não mostrará o número 5. Vejamos na **Figura 59**:

Problems | Tasks | Console | Properties

```
<terminated> Laboratorio.exe [C/C++ Application] D:\cpp\workspa
1 2 3 4 6 7 8 9 10
```

Figura 59. Resultado da execução do programa com continue

Vejamos agora esse mesmo exemplo trocando o comando continue pelo break, para notarmos a diferença de um para o outro.

```c
// Escrever os números de 1 a 10
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i;
    for (i = 1; i <= 10; i++) {
        if (i == 5) {
            break;
        }
        printf("%d ", i);
    }
    return 0;
}
```

Figura 60. Exemplo trocando continue pelo break

O que acontecerá, nesse caso, é que no momento em que a variável de controle do for, i, assumir valor 5, o teste da linha 8 novamente será verdadeiro e o comando da linha 9 será executado. Como agora se trata de um break, a repetição será terminada e a execução sairá do for. Sendo assim, a sequência exibida irá somente até o número 4, como na **Figura 60**, a seguir:

```
Problems  Tasks  Console  Properties
<terminated> Laboratorio.exe [C/C++ Application] D:\cpp\wor
1 2 3 4
```

Figura 61. Resultado da execução

Exercícios Propostos

1. (Resolvido) Faça um programa que o usuário entre com cinco números e imprima a metade de cada um deles.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>

int main() {
    int n;
    int i = 1;
    do {
        printf("Digite %do. o número: ", i);
        fflush(stdout);
        scanf("%d", &n);
        printf("Metade: %d\n", n/2);
        if (i == 5) {
            break;
        }
        i++;
    } while (1);
    return 0;
}
```

Execução:

Console

<terminated> Laboratorio.exe [C/C++ App

```
Digite 1o. o número: 8
Metade: 4
Digite 2o. o número: 10
Metade: 5
Digite 3o. o número: 12
Metade: 6
Digite 4o. o número: 20
Metade: 10
Digite 5o. o número: 32
Metade: 16
```

2. Escreva um programa que imprima na tela os números de 100 a 600 que são múltiplos de 5 e de 3 (os dois ao mesmo tempo).

3. Escreva um programa onde o usuário diz quantos números quer digitar, em seguida solicite a ele que digite todos os números e diga qual o menor número daqueles digitados.

4. Faça um programa que calcule a multiplicação de dois números através de somas sucessivas.

5. Entrar com 20 números e imprimir a soma dos números cujos quadrados são menores do que 225.

6. Ler uma determinada quantidade de números e dizer quantos são pares e quantos são ímpares.

7. Entrar com um número (n) e imprimir na tela o valor de H de tal modo que:
H=1+1/2+1/3+1/4+...+1/n
8. Entrar com um número (n) e imprimir na tela o valor de H de tal modo que:
H=1-1/2+1/3+1/4+...+1/n
9. Repare a seguinte característica do número 3025: 30 + 25 = 55 e 55**2 = 3025.
Criar um programa que possa ler vários números inteiros de 4 algarismos, um de cada vez, e diga se o número apresenta a mesma característica (repare que 3025/100=30 com resto 25). O algoritmo para quando for lido um valor menor que 1000 ou maior que 9999.
10. Criar um programa que recebe vários números e imprima o produto dos ímpares e a soma dos pares.
11. Desafio - um marciano chegou a uma floresta e se escondeu atrás de uma das 100 árvores quando viu um caçador. O caçador só tinha cinco balas em sua espingarda. Cada vez que ele atirava, e não acertava, é claro, o marciano dizia: “estou mais à direita ou mais à esquerda”. Se o caçador não conseguir acertar o marciano, ele será levado para marte. Implementar este jogo para dois jogadores, em que um escolhe a árvore onde o marciano irá se esconder e o outro tenta acertar.

Resumindo



Nesta unidade abordamos os comandos de seleção e de repetição em C, comandos essenciais no aprendizado de qualquer linguagem de programação. Vimos que os comandos *if, if-else* e *switch* nos dão a possibilidade de escolher uma determinada parte do código para ser executada ou não, e que os comandos for, *while* e *do-while* são responsáveis por executar uma mesma parte do código várias vezes consecutivas.

UNIDADE 03

Comandos de Seleção e de Repetição

3 COMANDOS DE SELEÇÃO E DE REPETIÇÃO

Vetores

Um vetor (array) é uma estrutura homogênea que serve para armazenarmos diversos valores do mesmo tipo. Por exemplo, se em nosso programa quisermos armazenar as notas da primeira prova dos 30 alunos da turma de Laboratório de Programação, não precisamos criar diversas variáveis (aluno 1, aluno 2, ...,aluno 30), basta usarmos um vetor que teremos todas as notas em uma só estrutura mais fácil de manipular. A sintaxe para declaração de um vetor é a seguinte:

tipo nome_da_variável[quantidade_de_posições]

Um vetor pode guardar diversos valores do mesmo tipo, no entanto, devemos especificar previamente quantos valores ele terá, através de um número inteiro (quantidade_de_posições) entre colchetes. Ao declararmos um vetor, estamos reservando uma quantidade de posições consecutivas em memória para armazenar esses valores. Vejamos na Figura 62, a seguir, um esquema de representação da memória para um vetor:



Figura 62. Representação em memória de um vetor de 4 posições

Com a declaração de um vetor, criamos uma sequência de elementos no qual podemos armazenar vários valores e acessá-los facilmente da seguinte maneira:

nome_da_variável[posição]

A posição indicará qual dos espaços de memória será acessado para obtenção do conteúdo. Vejamos, no exemplo a seguir (Figura 63), a declaração e utilização de um vetor na prática:

Laboratorio.c

```
1  // Utilização de vetor
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void) {
6      int idade[5];
7      idade[0] = 24;
8      idade[1] = 20;
9      idade[2] = 21;
10     idade[3] = 12;
11     idade[4] = 32;
12     printf("Idades: ");
13     printf("%d, ", idade[0]);
14     printf("%d, ", idade[1]);
15     printf("%d, ", idade[2]);
16     printf("%d, ", idade[3]);
17     printf("%d, ", idade[4]);
18     return 0;
19 }
```

Figura 63. Exemplo de declaração e utilização de vetor

Declaramos um vetor de inteiros com 5 posições (linha 6). Para cada posição atribuímos um valor, também do tipo inteiro (linhas 7 a 11). Por fim, acessamos cada posição do vetor na hora de escrever no printf da linha 12 quando colocamos cada posição. Devemos observar que o índice inicial de um vetor é 0, portanto, as posições acessíveis para um vetor de 4 posições declarado como int vetor[4] são 0, 1, 2 e 3.

É fácil notar, ainda, que temos uma forma mais prática de reescrever essa linha 12 através de um comando de repetição. Vejamos essa transposição na Figura 64, a seguir:

Laboratorio.c

```
1  // Utilização de vetor
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void) {
6      int idade[5];
7      idade[0] = 24;
8      idade[1] = 20;
9      idade[2] = 21;
10     idade[3] = 12;
11     idade[4] = 32;
12     int i;
13     printf("Idades: ");
14     for (i = 0; i < 5; i++) {
15         printf("%d, ", idade[i]);
16     }
17     return 0;
18 }
```

Figura 64. Reescrita do exemplo usando for

Assim, temos uma forma prática e direta de manipular um conjunto de valores. Através do for da linha 14, percorremos todas as posições do vetor e acessamos os valores de cada uma das posições através da variável i. A seguir (Figura 65) temos o resultado da execução deste programa.

Console

```
<terminated> Laboratorio.exe [C/C++ Application] D:\cpp
Idades: 24, 20, 21, 12, 32
```

Figura 65. Resultado da execução

Da mesma forma que usamos o for para escrever os valores do vetor, podemos também usar um for (ou while, ou do-while) para preencher os valores desse vetor. Nesse caso, podemos aproveitar para pedir para o usuário digitar os valores. Vejamos o mesmo exemplo anterior modificado para que a entrada de valores seja efetuada pelo usuário através de um for, na Figura 66 a seguir:

Laboratorio.c

```
1 // Utilização de vetor
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void) {
6     int idade[5];
7     int i;
8     for (i = 0; i < 5; i++) {
9         printf("Digite a %da. idade: ", i + 1);
10        fflush(stdout);
11        scanf("%d", &idade[i]);
12    }
13    printf("Idades: ");
14    for (i = 0; i < 5; i++) {
15        printf("%d, ", idade[i]);
16    }
17    return 0;
18 }
```

Figura 66. Utilização do for para preenchimento do vetor

Vejamos que na linha 8, o for que usamos para preencher é exatamente igual ao de escrita. Na linha 9 usamos os índices i do vetor para pedir cada posição ao usuário. Somamos com 1 para que não comece em 0, mas em 1. Na linha 11 fazemos a leitura dos valores para o vetor, passando para o scanf a posição do vetor (&idade[i]). Um possível resultado para a execução deste programa encontra-se na Figura 67 abaixo.

Console

```
<terminated> Laboratorio.exe [C/C++ Application]
Digite a 1a. idade: 10
Digite a 2a. idade: 20
Digite a 3a. idade: 23
Digite a 4a. idade: 15
Digite a 5a. idade: 29
Idades: 10, 20, 23, 15, 29,
```

Figura 67. Resultado da execução

Exercícios Propostos

1. Faça um programa que leia 10 valores reais em um vetor e, logo após, calcule a média desses valores e a imprima para o usuário.
2. Faça um programa que leia um vetor de 10 valores e imprima o menor.
3. Faça um programa que leia 10 valores em um vetor. Depois de ler o

vetor, o programa deve percorrê-lo e imprimir apenas os valores ímpares que foram armazenados.

4. Faça um programa que preencha 2 vetores, com 5 elementos cada. Como resultado, o programa deve apresentar um vetor com os elementos dos dois vetores intercalados. Vejamos um exemplo a seguir:

- Vetor 1: [5, 8, 10, 3, 1]
- Vetor 2: [4, 2, 9, 7, 6]
- Vetor Resultado: [5, 4, 8, 2, 10, 9, 3, 7, 1, 6].

5. (Resolvido) Faça um programa que receba 10 valores para um vetor e o imprima ao contrário.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>

int main() {
    int n[10], i;
    for (i = 0; i < 10; i++) {
        printf("Digite %do. o número: ", i + 1);
        fflush(stdout);
        scanf("%d", &n[i]);
    }
    for (i = 9; i >= 0; i--) {
        printf("%d, ", n[i]);
    }
    return 0;
}
```

Execução:

Console

```
<terminated> Laboratorio.exe [C/C++ Application] D:
Digite 1o. o número: 2
Digite 2o. o número: 4
Digite 3o. o número: 5
Digite 4o. o número: 6
Digite 5o. o número: 7
Digite 6o. o número: 8
Digite 7o. o número: 99
Digite 8o. o número: 10
Digite 9o. o número: 12
Digite 10o. o número: 15
15, 12, 10, 99, 8, 7, 6, 5, 4, 2,
```

6. Faça um programa que receba dez números inteiros e armazene-os em um vetor. O programa deve calcular e mostrar dois vetores resultantes, sendo o primeiro com os números pares e o segundo com os números ímpares do vetor lido.

7. Faça um programa que receba dois vetores de 5 elementos cada e retorne um vetor resultado contendo a soma das posições correspondentes dos vetores de entrada. Por exemplo:

- Vetor 1: [5, 8, 10, 3, 1]
- Vetor 2: [4, 2, 9, 7, 6]
- Vetor Resultado: [9, 10, 19, 10, 7].

8. Faça um programa que leia um vetor de 10 elementos. Logo em seguida leia um valor a ser encontrado nesse vetor. Pesquise o valor no vetor e retorne a posição onde o valor está localizado. Se não for encontrado escreva a mensagem: valor não encontrado.

- Vetor: [10, 8, 4, 2, 7, 12, 19, 21, 37, 18]
- Valor a ser procurado: 18
- Resultado: Encontrado na posição: 9.

9. Leia dois vetores: R de 5 elementos e S de 10 elementos. Gerar um vetor X de 15 elementos cujas 5 primeiras posições contenham os elementos de R e as 10 últimas posições os elementos de S. Escrever o vetor X.

10. Faça um programa que receba 10 valores em um vetor e os coloque em ordem crescente. Serão necessários dois percorrimentos sobre os elementos desse vetor, para pesquisar cada elemento e depois para verificar se são menores que os já verificados. Veja a seguir:

```
para i de 0 até tamanho[vetor] - 1
        para j de i até tamanho[vetor]
                se vetor[i] > vetor[j]
                        aux = vetor[i]
                        vetor[i] = vetor[j]
                        vetor[j] = aux
```

Strings ou cadeias de caracteres (vetores de char)

Conhecendo a estrutura de vetores e sabendo da existência do tipo char, que pode armazenar apenas um caractere, podemos deduzir facilmente o que são strings. Através da construção de um vetor de char, representamos, em C, palavras ou frases, enfim, cadeias de caracteres. A declaração de uma string segue o mesmo formato de um vetor convencional:

char nome_da_variavel[quantidade_de_posições]

A quantidade de posições deve ser tal qual possa comportar a palavra que queremos armazenar, ou seja, esse valor quantidade_de_ posições especificado indica o tamanho máximo da nossa string - 1. Tamanho subtraído de 1, porque devemos sempre deixar uma posição a mais, visto que ao final de toda string haverá sempre o caractere nulo, como na representação a seguir (Figura 68).

Laboratorio.c

```
1 // Utilização de string
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void) {
6     char nome[50];
7     printf("Digite seu nome: ");
8     fflush(stdout);
9     gets(nome);
10    printf("Seu nome é: %s", nome);
11    return 0;
12 }
```

Figura 68. Representação de uma string

Mesmo que todas as posições não sejam preenchidas, o caractere nulo será armazenado logo após o último caractere de nossa palavra, ou seja, é ele quem indicará o final da string, independente do vetor estar todo preenchido ou não.

Para fazermos a leitura de uma variável do tipo string, usamos uma função especial ao invés do scanf, chamada gets. No exemplo a seguir (Figura 69) veremos tanto a declaração de string quanto a utilização do método gets para leitura das mesmas.

Laboratorio.c

```
1 // Utilização de string
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void) {
6     char nome[50];
7     printf("Digite seu nome: ");
8     fflush(stdout);
9     gets(nome);
10    printf("Seu nome é: %s", nome);
11    return 0;
12 }
```

Figura 69. Utilização de strings

Nesse exemplo declaramos nossa string na linha 6 e usamos a função gets para ler um valor do tipo string e armazenar nessa variável. Por isso temos que passar nossa variável para a função gets, para que ele possa atribuir o valor lido do teclado à variável especificada em sua declaração.

Um possível resultado para a execução deste programa está logo abaixo na Figura 70:

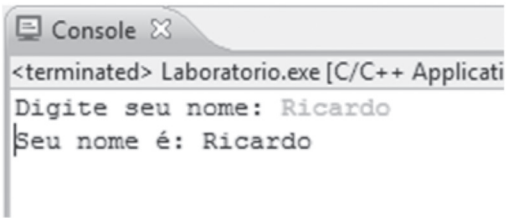


Figura 70. Resultado da execução do programa

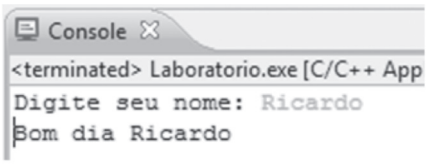
Exercícios Propostos

1. (Resolvido) Faça um programa que receba seu nome e escreva bom dia para você.

Laboratorio.c

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 int main() {
5     char nome[20];
6     printf("Digite seu nome: ");
7     fflush(stdout);
8     gets(nome);
9     printf("Bom dia %s", nome);
10    return 0;
11 }
```

Execução:



2. Faça um programa que receba seu nome e sobrenome em variáveis separadas e imprima seu nome completo numa só linha.
3. Faça um programa que receba um nome e o imprima ao contrário.
4. Faça um programa que receba um nome e conte quantos caracteres possui a palavra recebida.
5. Faça um programa que receba duas strings, percorra as duas comparando cada um dos seus elementos e, ao final, imprima se as duas são iguais ou não.

Funções de string

Existem funções específicas para trabalharmos com strings. Como não são variáveis simples, os operadores que conhecemos até agora não funcionam com este tipo. Podemos fazer muitas coisas com strings, como, por exemplo, ver seu tamanho, juntar mais de uma palavra, comparar duas palavras diferentes, etc. Estas funções estão na biblioteca string.h. Sendo assim, para que possamos utilizá-las, devemos incluir esta biblioteca através da declaração:

```
#include <string.h>
```

Na Tabela 6, a seguir, temos as principais funções com seu significado:

Função	Significado	Exemplo
Strlen (string)	Tamanho da string	strlen (nome)
strcmp (string1, string2)	Comparação entre duas strings (0: iguais; diferente de 0 caso contrário)	strcmp(nome, endereço)
strcpy(string1, string2)	Cópia do conteúdo de string2 em string1	strcpy(nome, endereço)
strcat(string1, string2)	Concatena 2 string: junta string2 no final de string1	strcat(nome, endereço)

Tabela 6. Funções para trabalhar com strings

É extremamente importante para a programação lidar com strings, pois elas são a base de qualquer arquivo e principalmente controles de rotina (por exemplo: rotinas e procedimentos de banco de dados, o SQL; endereçamento de arquivos; modificação de configurações...). A seguir, Figura 71, temos um exemplo com todas as funções básicas de manipulação de string vistas até aqui:

Laboratorio.c

```
1  // Utilização de funções string
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  int main(void) {
7      char palavra1[75], palavra2[25], palavraConcatenada[100];
8      printf("Digite uma palavra: ");
9      fflush(stdout);
10     gets(palavra1);
11     printf("Digite outra palavra: ");
12     fflush(stdout);
13     gets(palavra2);
14     printf("String-1: %s tem %d caracteres\n", palavra1, strlen(palavra1));
15     printf("String-2: %s tem %d caracteres\n", palavra2, strlen(palavra2));
16     if (strcmp(palavra1, palavra2) == 0) {
17         printf("%s é igual a %s\n", palavra1, palavra2);
18     } else {
19         printf("%s é diferente de %s\n", palavra1, palavra2);
20     }
21     strcpy(palavraConcatenada, palavra1);
22     printf("Palavra concatenada: %s \n", palavraConcatenada);
23     strcat(palavraConcatenada, palavra2);
24     printf("Palavra concatenada: %s \n", palavraConcatenada);
25     return 0;
26 }
```

Figura 71. Utilização das funções de string

Na Figura 72, a seguir, temos um possível resultado da execução deste programa:

Console

```
<terminated> Laboratorio.exe [C/C++ Application] D:\cpp\workspace\
Digite uma palavra: flamengo
Digite outra palavra: fluminense
String-1: flamengo tem 8 caracteres
String-2: fluminense tem 10 caracteres
flamengo é diferente de fluminense
Palavra concatenada: flamengo
Palavra concatenada: flamengofluminense
```

Figura 72. Resultado da execução do programa

Exercícios Propostos

1. (Resolvido) Faça um programa que receba um nome e compare com o seu. Se for igual escreva “NOME CORRETO”, caso contrário, escreva “NOME INCORRETO”.

Laboratorio.c

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

int main() {
    char nome[20];
    printf("Digite seu nome: ");
    fflush(stdout);
    gets(nome);
    if (strcmp(nome, "Ricardo") == 0) {
        printf("Nome correto");
    } else {
        printf("Nome incorreto");
    }
    return 0;
}
```

Execução:

Console

```
<terminated> Laboratorio.exe [C/C++ App
Digite seu nome: Ricardo
Nome correto
```

2. Faça um programa que receba duas palavras, concatene as duas e conte quantos caracteres possui o resultado dessa junção. Imprima a palavra concatenada e a quantidade de caracteres da concatenação.
3. Faça um programa que receba duas strings e troque os valores dessas variáveis. Imprima os novos valores das duas strings.
4. Faça um programa que receba uma string e conte quantas letras maiúsculas existem numa string recebida como parâmetro.
5. Faça um programa que receba uma string e conte quantas letras minúsculas existem numa string recebida como parâmetro.
6. Faça um programa que receba uma string e troque todas as letras maiúsculas por minúsculas e as minúsculas por maiúsculas de uma string recebida como parâmetro.
7. Escreva um programa em C para ler uma frase e contar o número de palavras existentes na frase. Considere palavra um conjunto qualquer de

caracteres separados por um conjunto qualquer de espaços em branco.
8. Escreva um programa em C para ler um caractere e logo após um número indeterminado de frases. Para cada frase informada imprimir o número de ocorrências do caractere na frase. O programa deve ser encerrado quando a frase digitada for a palavra "fim".
9. Faça um programa que leia uma string do teclado e diga se ela é palíndromo. Uma string é palíndromo quando pode ser lida tanto de trás pra frente quanto de frente para trás e possui exatamente a mesma sequência de caracteres. Ex.: ASA, SUBI NO ONIBUS. Desconsidere os espaços.
10. Faça um programa que receba 3 nomes e os coloque em ordem alfabética. Dica: use a função *strcmp*.

Matrizes

Podemos definir uma matriz como uma extensão dos vetores para mais dimensões. Ou ainda, redefinir um vetor como sendo uma matriz unidimensional. Sendo assim, a definição de matriz é a mesma de vetor, uma estrutura que armazena uma sequência de valores do mesmo tipo. No entanto, as matrizes podem ter 2, 3 ou mais dimensões. A declaração de uma matriz é feita da maneira a seguir:

tipo nome_da_variável[posições1][posições2]...[posiçõesn]

Dessa forma, podemos declarar nossa matriz com a quantidade de dimensões que quisermos, sendo mais natural e compreensível a utilização de matrizes com 2 ou 3 dimensões. Vejamos a seguir (Figura 73) um exemplo de declaração e utilização de uma matriz com 2 dimensões.

Laboratorio.c

```
1 // Utilização de matrizes
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void) {
6     float notas[4][3];
7     int i, j;
8     for (i = 0; i < 4; i++) {
9         for (j = 0; j < 3; j++) {
10            printf("Digite a %da. nota do %do. aluno: ", j + 1, i + 1);
11            fflush(stdout);
12            scanf("%f", &notas[i][j]);
13        }
14    }
15    for (i = 0; i < 4; i++) {
16        float media = (notas[i][0] + notas[i][1] + notas[i][2]) / 3;
17        printf("Média do %do. aluno: %.1f\n", i, media);
18    }
19    return 0;
20 }
```

Figura 73. Utilização de matriz com 2 dimensões

Nesse exemplo temos uma matriz bidimensional declarada na linha 6, com 4 linhas e 3 colunas: o primeiro número indica a quantidade de linhas e o segundo a quantidade de colunas. Para iniciar essa matriz, usamos dois comandos iterativos for aninhados nas linhas 8 e 9 (um dentro do outro), sendo que um servirá para percorrer as linhas e o outro, dentro de cada linha, para percorrer as colunas. Para acessar cada posição usamos uma estrutura idêntica a de vetores, com a diferença de colocarmos as 2 componentes, já que se trata de uma matriz:

nome_da_variavel[linha][coluna]

Cada componente terá seus colchetes correspondentes. Após as leituras de todas as notas, usamos outro laço for para calcular e imprimir as médias na linha 15. Nesse instante, usamos apenas uma iteração, pois, a cada passo, iremos acessar as três notas de cada aluno para o cálculo da média, linha 16. Para imprimir a média usamos o printf normalmente com a formatação "%.1f" que indicará a escrita de apenas uma casa decimal.

Poderíamos ter iniciado a matriz (vetores também) no momento da declaração, assim como fazemos com variáveis comuns. A sintaxe para isso, para uma matriz bidimensional, por exemplo, é a seguinte:

tipo nome_da_variavel[linhas][colunas] = {{elementos_linha1}, {elementos_linha2}, ... }

Para ficar mais claro, vejamos o mesmo exemplo anterior usando esse tipo de declaração na Figura 74, a seguir:

Laboratorio.c

```
1  // Utilização de matrizes
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void) {
6      float notas[4][3] = { {9.0, 8.0, 7.0}, {6.5, 9.5, 7.0},
7                            {4.5, 3.5, 9.0}, {9.5, 9.7, 9.3} };
8      int i;
9      for (i = 0; i < 4; i++) {
10         float media = (notas[i][0] + notas[i][1] + notas[i][2]) / 3;
11         printf("Média do %do. aluno: %.1f\n", i + 1, media);
12     }
13     return 0;
14 }
```

Figura 74. Inicialização de uma matriz

Esta forma de iniciação pode ser usada também para vetores, onde, logicamente, teremos apenas uma componente ao invés de duas. Se tivéssemos uma matriz tridimensional teríamos três componentes e assim por diante. A seguir, Figura 75, temos uma possível saída para o exemplo de utilização de matrizes visto.

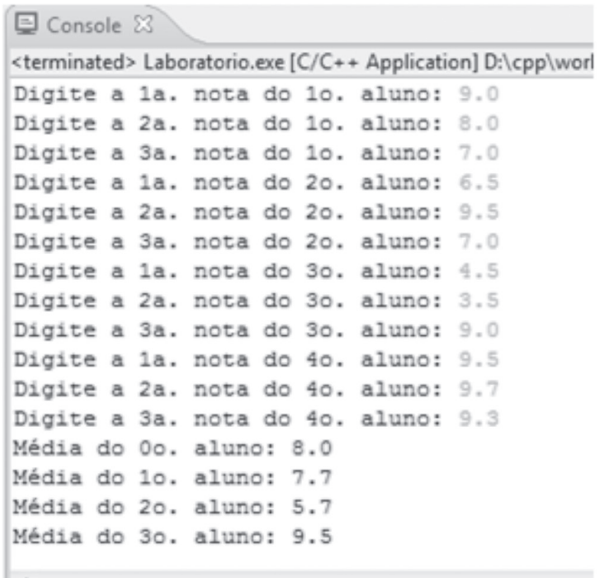


Figura 75. Possível saída para o programa

Para o exemplo com a iniciação da matriz na própria declaração, temos a seguinte saída (Figura 76):

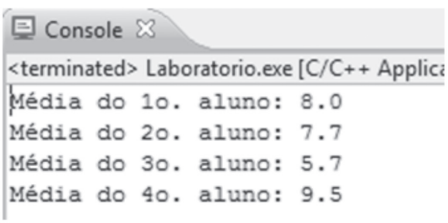


Figura 76. Saída do programa com iniciação da matriz

Como podemos armazenar quaisquer tipos de valores nas matrizes, logicamente também podemos armazenar strings. Como strings já são vetores por definição na linguagem C, temos algumas particularidades na utilização de matrizes de strings. Analisemos o exemplo da Figura 77, a seguir:

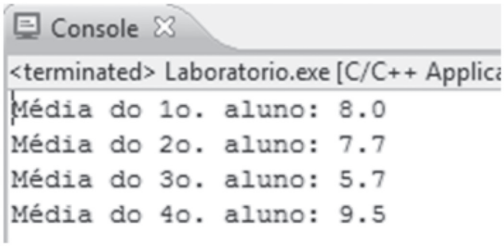


Figura 77. Utilização de vetores de matrizes

Para declarar uma matriz de strings, usamos o mesmo modo de declaração de matrizes com o tipo char, como podemos ver na linha 6. A diferença maior está no modo de acesso a cada string, ao invés de passarmos os 2 índices para acesso à posição, passamos apenas 1, indicando a linha onde está armazenada a string. Dessa forma, será retornada a palavra armazenada naquela linha da matriz (linha 11 e linha 15).

Se quisermos inicializar uma matriz desse tipo no momento da declaração, faremos da mesma forma vista anteriormente, no entanto, para cada linha colocaremos a string correspondente entre aspas duplas(“), ao invés de entre chaves com cada elemento separado por vírgula quando usamos o tipo float, por exemplo. Vejamos, a seguir (Figura 78), como ficaria o programa anterior iniciando a matriz no momento da declaração;

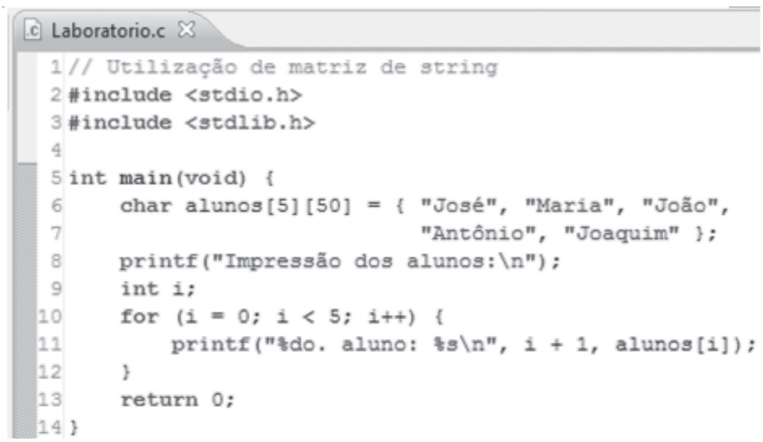


Figura 78. Iniciação de uma matriz de strings

Na Figura 79 temos a saída para este programa. Percebam que tanto na leitura, através do gets, quanto na escrita, pelo *printf*, o acesso

à string se dá apenas por um índice.

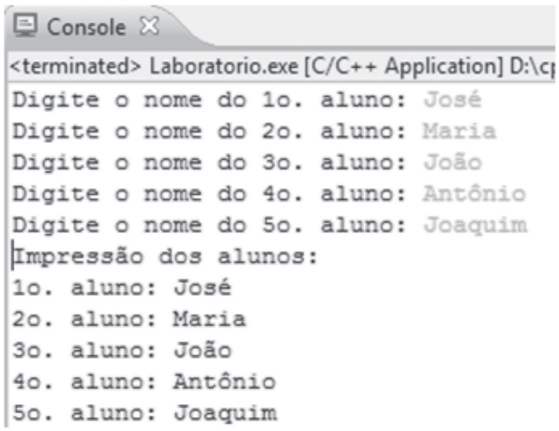


Figura 79. Possível saída do programa com matrizes de strings

Exercícios Propostos

1. Faça um programa que receba os elementos de uma matriz e imprima a soma de todos os seus elementos.
2. Faça um programa que receba os elementos de uma matriz 4x4. Exiba a matriz e verifique se ela é a identidade. Na figura a seguir temos um exemplo de matriz identidade para dimensão 2x2.
3. (Resolvido) Faça um programa que receba os elementos de uma matriz 4x4. Depois receba um valor do mesmo tipo dos elementos da matriz. Verifique e imprima quantas vezes esse valor existe na matriz. Se não encontrar imprima “não existe”.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

int main() {
    int n[4][4], i, j, num, encontrado = 0;
    printf("Digite os elementos da matriz:\n");
    for (i = 0; i < 4; i++) {
        for ( j = 0; j < 4; j++) {
            printf("Elemento [%d][%d]: ", i + 1, j + 1);
            fflush(stdout);
            scanf("%d", &n[i][j]);
        }
    }
    printf("Digite o número a encontrar: ");
    fflush(stdout);
    scanf("%d", &num);
    for (i = 0; i < 4; i++) {
        for ( j = 0; j < 4; j++) {
            if (n[i][j] == num) {
                encontrado++;
            }
        }
    }
    printf("Número %d encontrado %d vezes na matriz", num, encontrado);
    return 0;
}
```

Execução:

Console

```
<terminated> Laboratorio.exe [C/C++ Application] D:\cpp\
Digite os elementos da matriz:
Elemento [1][1]: 1
Elemento [1][2]: 2
Elemento [1][3]: 3
Elemento [1][4]: 4
Elemento [2][1]: 5
Elemento [2][2]: 1
Elemento [2][3]: 2
Elemento [2][4]: 3
Elemento [3][1]: 1
Elemento [3][2]: 2
Elemento [3][3]: 3
Elemento [3][4]: 4
Elemento [4][1]: 3
Elemento [4][2]: 2
Elemento [4][3]: 1
Elemento [4][4]: 2
Digite o número a encontrar: 2
Número 2 encontrado 5 vezes na matriz
```

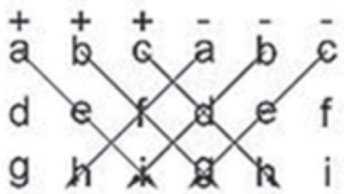
4. Faça um programa que receba os elementos de uma matriz. Exiba a matriz e sua transposta. Na matriz transposta trocam-se linhas por colunas e colunas por linhas da matriz original. Veja a figura a seguir:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, A^t = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

5.Faça um programa que receba uma matriz e verifique se ela é uma matriz simétrica. Uma matriz simétrica significa que ela é igual à sua transposta.

6.Faça um programa que receba uma matriz 5x5 e verifique se essa matriz é triangular. Uma matriz triangular possui todos os elementos abaixo ou acima da diagonal principal são iguais a zero.

7.Faça um programa que receba uma matriz 3x3 e calcule seu determinante. O determinante de uma matriz 3x3 é calculado através de suas diagonais, conforme a figura a seguir:



aei + bfg + cdh - afh - bdi - ceg

8.Faça um programa que receba uma matriz 4x4 e uma constante e imprima a matriz resultante da multiplicação da matriz pela constante.

9.Faça um programa que receba duas matrizes 4x4 e imprima o resultado da soma das duas matrizes.

10.Faça um programa que multiplique duas matrizes. Não é necessário pedir para o usuário digitar os valores, ou seja, declare as matrizes iniciadas no momento da declaração. Para que seja possível a multiplicação, o número de colunas da primeira deve ser igual ao número de linhas da segunda. O cálculo da multiplicação faz-se como segue:

$$\begin{bmatrix} 1 & 0 & 2 \\ -1 & 3 & 1 \end{bmatrix} \times \begin{bmatrix} 3 & 1 \\ 2 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} (1\times3+0\times2+2\times1) & (1\times1+0\times1+2\times0) \\ (-1\times3+3\times2+1\times1) & (-1\times1+3\times1+1\times0) \end{bmatrix} = \begin{bmatrix} 5 & 1 \\ 4 & 2 \end{bmatrix}$$

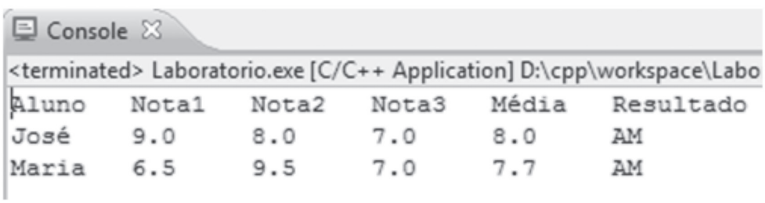
11.Faça um programa que receba cinco nomes em uma matriz de strings e os exiba, todos na mesma linha, concatenados através de um for (ou while, ou do-while).

12.Faça um programa que receba dez nomes em uma matriz de strings e outra string seq em separado. Imprima as palavras pertencentes à matriz que contenham a string seq recebida. Por exemplo, a string atestada contém a string testa, então deve ser impressa.

13.Faça um programa que receba dez nomes em uma matriz de strings e imprima os nomes que forem repetidos.

14.Faça um programa que receba dez nomes em uma matriz de strings e os imprima em ordem alfabética.

15.Faça um programa que contenha duas matrizes, uma com as três notas de cada aluno e outra com os nomes dos alunos de determinada turma. Ao final imprima o relatório da turma com os nomes de cada aluno, seguido das respectivas notas, da média e do resultado final, sendo aprovado (AM) para média >= 7, prova final (EF) para 4 <= média < 7 e reprovado (RP) para média < 4. Por exemplo:





Resumindo

Nessa unidade vimos como agrupar diversos dados em uma mesma estrutura, os vetores (e matrizes) por meio dos quais podemos armazenar, em sequência, diversos elementos do mesmo tipo e, posteriormente, acessá-los de forma simples e rápida. Abordamos um vetor especial, o vetor de char, também chamado de string, através do qual podemos manipular cadeias de caracteres na linguagem de programação C.

UNIDADE 04

Estruturas

4 ESTRUTURAS

Estruturas

Usar uma estrutura significa criar um tipo de dado que armazene vários membros de tipos de dados diferentes. A rigor, criaremos variáveis que possuem, como componentes, outras variáveis de tipos distintos. A grande vantagem disso é a organização de nossos programas, por exemplo, quando quisermos armazenar os campos de um aluno como nome, matrícula, notas, etc. não precisamos mais criar variáveis em separado. Dentro da mesma estrutura agruparemos todos os dados referentes a um aluno.

A especificação de um tipo de dados estrutura se dá através da palavra chave struct. As variáveis dentro da struct se chamam membros. Vejamos o formato de declaração:

```
struct nome_da_estrutura {
        tipo_do_membro1 nome_do_membro1;
        tipo_do_membro2 nome_do_membro2;
        ...
        tipo_do_membro2 nome_do_membro2;
} variáveis_estrutura;
```

Através dessa declaração temos nossa estrutura com os membros descritos. Por exemplo, temos na Figura 80 um exemplo de declaração de estrutura data, possuindo os campos dia, mês e ano. No entanto, não temos nenhuma variável declarada.

Laboratorio.c

```
1 // Declaração de estrutura
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 struct data {
6     int dia;
7     int mes;
8     int ano;
9 };
10
11 int main(void) {
12     return 0;
13 }
```

Figura 80. Declaração de struct para armazenar uma data

Nas linhas de 5 a 8 declaramos nossa estrutura. A partir dessa declaração, podemos criar variáveis que armazenem dados com esses três campos. Para fazermos isso, colocamos logo à frente da declaração da struct, após a chave de fechamento, o nome das variáveis que queremos declarar. A partir da variável podemos ter acesso aos campos através do operador ponto (.). Se declararmos uma variável de nome hoje e tipo data, o acesso aos campos se dá através das chamadas hoje. dia, hoje.mes, hoje.ano.

A utilização de estruturas possibilita o isolamento dos membros, ou seja, para cada variável do tipo data que criarmos, cada uma terá seus próprios valores para dia, mês e ano, possibilitando um gerenciamento organizado e evitando a mistura dos valores. Vejamos na Figura 81 um exemplo de declaração e uso de variáveis desse tipo.

Laboratorio.c

```
1 // Declaração e uso de estruturas
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 struct data {
6     int dia;
7     int mes;
8     int ano;
9 } hoje, amanha;
10
11 int main(void) {
12     hoje.dia = 10;
13     hoje.mes = 10;
14     hoje.ano = 2009;
15     amanha.dia = 11;
16     amanha.mes = 10;
17     amanha.ano = 2009;
18     printf("%d/%d/%d\n", hoje.dia, hoje.mes, hoje.ano);
19     printf("%d/%d/%d", amanha.dia, amanha.mes, amanha.ano);
20     return 0;
21 }
```

Figura 81. Exemplo de uso de structs

Aqui declaramos duas variáveis do tipo de nossa estrutura data: hoje e amanhã (linha 9). Na função main é possível verificar a chamada aos membros dessa estrutura. Nas linhas 12, 13 e 14 atribuímos valores para os membros da variável hoje. Nas linhas 15, 16 e 17 fazemos o mesmo para os membros da variável amanhã. Já nas linhas 18 e 19 fazemos uma chamada aos atributos para escrevermos seus valores na tela. Através da saída desse programa vista abaixo (Figura 82), verificamos que cada variável possui seus próprios dados, e uma não interfere no conteúdo da outra.

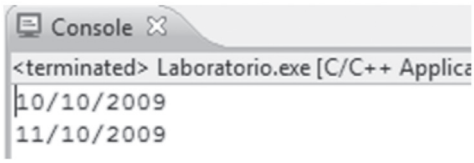


Figura 82. Resultado da execução

Declaração de tipos de dados estruturas

Percebemos, através dos exemplos anteriores, que há uma pequena limitação nessa forma de utilização. Todas as variáveis que precisarmos do tipo da estrutura devem ser declaradas no momento da especificação da estrutura. Para contornarmos isso, podemos declarar um tipo de dados através da palavra-chave typedef, da seguinte forma:

typedef especificação_do_tipo nome_do_tipo;

Utilizando esta declaração, criamos um novo tipo, personalizado, que será identificado através do identificador nome_do_tipo que colocamos como último componente dessa especificação. Dessa forma, podemos alterar nossos primeiros exemplos de struct incluindo a criação do tipo e, assim, podermos criar variáveis a qualquer momento em nosso programa, não somente na declaração da estrutura. Em outras palavras, utilizamos nosso tipo como qualquer dos outros tipos que já estudamos.

Por exemplo, vejamos a definição de um tipo de dados nome (Figura 83) usando a definição de string que vimos na unidade anterior:

Laboratorio.c

```
1  // Declaração e uso de estruturas
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  typedef char nome[50];
6
7  int main(void) {
8      nome aluno = "Ricardo";
9      printf("%s", aluno);
10     return 0;
11 }
```

Figura 83. Criação de um tipo de dados

Através da declaração na linha 5 criamos o novo tipo nome. Para criarmos nossa variável aluno usamos, agora, nossa nova definição e não mais o vetor de char (linha 8). Com isso, damos mais expressividade ao nosso código, facilitando a leitura e a interpretação do mesmo. Mas, cuidado, não devemos redefinir tipos básicos da linguagem, como int, float, char, etc. pois dessa forma acabamos obtendo o efeito contrário, ou seja, prejudicando a leitura do nosso código.

Para declararmos um novo tipo com a definição de uma estrutura, seguimos o mesmo princípio. Vejamos no exemplo a seguir (Figura 84) como é feita. Usaremos a mesma estrutura para datas vista anteriormente:

Laboratorio.c

```
1  // Declaração e uso de estruturas
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  typedef struct data {
6      int dia;
7      int mes;
8      int ano;
9  } tipoData;
10
11 int main(void) {
12     tipoData hoje, amanha;
13     hoje.dia = 10;
14     hoje.mes = 10;
15     hoje.ano = 2009;
16     amanha.dia = 11;
17     amanha.mes = 10;
18     amanha.ano = 2009;
19     printf("%d/%d/%d\n", hoje.dia, hoje.mes, hoje.ano);
20     printf("%d/%d/%d", amanha.dia, amanha.mes, amanha.ano);
21     return 0;
22 }
```

Figura 84. Especificação de tipo a partir de uma struct

Usando a definição de tipo, o nome que precede a definição da estrutura, logo após a chave de fechamento (tipoData, na linha 9), será o nome que usaremos para nos referenciar a este tipo. Sendo assim, podemos declarar variáveis em qualquer lugar do nosso programa através do nome especificado (tipoData), como foi feito na linha 12.

Poderíamos ainda separar a definição da estrutura da declaração do tipo. No exemplo abaixo (Figura 85) temos a definição feita dessa forma.



Figura 85. Definição do tipo separada da definição da estrutura

Os dois códigos têm rigorosamente o mesmo significado, sendo assim, fica a critério do programador escolher entre uma ou outra forma de declaração. Inclusive o resultado da execução desses programas permanece o mesmo já mostrado na Figura 82.



Exercícios Propostos

1. (Resolvido) Faça um programa que defina uma estrutura com os dados de um carro, a saber: modelo (string), fabricante (string), ano (inteiro), quantidade de portas (inteiro), potência do motor (float), tipo de combustível (string), vidro elétrico (bool), trava elétrica (bool), ar condicionado (bool), direção hidráulica (bool).

Laboratorio.c

```c
// Declaração e uso de estruturas
#include <stdio.h>
#include <stdlib.h>

typedef struct data {
    int dia;
    int mes;
    int ano;
} tipoData;

int main(void) {
    tipoData hoje, amanha;
    hoje.dia = 10;
    hoje.mes = 10;
    hoje.ano = 2009;
    amanha.dia = 11;
    amanha.mes = 10;
    amanha.ano = 2009;
    printf("%d/%d/%d\n", hoje.dia, hoje.mes, hoje.ano);
    printf("%d/%d/%d", amanha.dia, amanha.mes, amanha.ano);
    return 0;
}
```

2. Faça um programa que possua uma estrutura pessoa com os seguintes membros: nome do tipo string, endereço do tipo string, bairro do tipo string, cidade do tipo string, estado do tipo string, CEP do tipo string, idade do tipo inteiro, altura do tipo float e peso do tipo float. Seu programa deverá possuir as seguintes funções:

a. Cadastrar pessoa: com leitura do teclado de todos os dados da pessoa;

b. Mostrar pessoa: com escrita na tela de todos os dados correspondentes à pessoa cadastrada;

c. Alterar pessoa: escolha um dos campos a serem alterados na pessoa cadastrada e coloque um novo valor nesse campo, mostrando depois todos os campos para que se possa verificar o valor alterado.

3. Faça um programa que contenha uma estrutura aluno com os seguintes membros: nome do tipo string, matrícula do tipo string, nota1 do tipo float, nota2 do tipo float e nota3 do tipo float. Seu programa deverá receber todos os dados de um aluno e ao final exibir o relatório do mesmo da seguinte forma:

Aluno nome de matrícula matrícula, possui notas: nota1, nota2 e nota3. Sendo assim, está (aprovado|reprovado|de prova final), com média media

4. Crie um programa que contenha uma estrutura retângulo com os seguintes membros: base de tipo float e altura de tipo float. Seu programa deverá pedir para o usuário entrar com os dados de base e altura e imprimir a área desse retângulo.

5. Faça um programa que contenha as definições de registros para os seguintes dados: (OBS: cada item corresponderá a uma estrutura).

a. Um veículo novo de uma concessionária de automóveis;

b. Um eletrodoméstico de uma loja;

c. Um prato em um restaurante.

6. Defina um tipo de registro para armazenar dados de um voo, como por exemplo os nomes das cidades de origem e de destino, datas e horários de partida e datas e horários de chegada. Seu programa deverá conter:

a. Uma opção para cadastrar o voo, lendo todos os dados que o usuário digitar;

b. Uma opção para alterar o voo cadastrado, alterando apenas o dado escolhido pelo usuário para alteração;

c. Uma opção para mostrar todos os dados do voo cadastrado.

Vetores de Estruturas

Podemos unir os dois conceitos estudados nessa unidade e na anterior. Ou seja, podemos criar vetores de estruturas e, assim, armazenar cadastros de vários elementos de dados do tipo da estrutura. Dessa

forma, podemos criar cadastros e gerenciar dados de várias entidades em nossos programas. Vejamos o exemplo a seguir (Figura 86).

Laboratorio.c

```
1  // Uso de vetores de estruturas
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  typedef struct data tipoData;
6
7  struct data {
8      int dia;
9      int mes;
10     int ano;
11 };
12
13 int main(void) {
14     tipoData dias[2];
15     dias[0].dia = 10;
16     dias[0].mes = 10;
17     dias[0].ano = 2009;
18     dias[1].dia = 11;
19     dias[1].mes = 10;
20     dias[1].ano = 2009;
21     printf("%d/%d/%d\n", dias[0].dia, dias[0].mes, dias[0].ano);
22     printf("%d/%d/%d", dias[1].dia, dias[1].mes, dias[1].ano);
23     return 0;
24 }
```

Figura 86. Exemplo de vetor de estruturas

Nesse exemplo temos a mesma aplicação anterior, no entanto, armazenamos as duas datas na mesma variável, que agora é um vetor, em duas posições diferentes. Acessamos as posições do vetor normalmente, da forma como vimos na unidade anterior e os campos de cada posição são acessados pelo operador ponto (.), como podemos visualizar nas linhas 15 a 20. O resultado da execução é o mesmo visto na Figura 82.

Podemos, também, utilizar vetores como membros de nossas estruturas. Por exemplo, se quisermos armazenar as três notas de um aluno, é melhor utilizarmos um vetor para isso do que três variáveis distintas. Também podemos utilizar strings que, a rigor, também significa utilizarmos vetores em nossa estrutura. Vejamos o exemplo da Figura 87 a seguir:

Laboratorio.c

```
1  // Uso de vetores de estruturas
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  typedef struct s_aluno tipoAluno;
7
8  struct s_aluno {
9      char nome[50];
10     float nota[3];
11 };
12
13 int main(void) {
14     tipoAluno aluno;
15     strcpy(aluno.nome, "Ricardo");
16     aluno.nota[0] = 9.0;
17     aluno.nota[1] = 8.5;
18     aluno.nota[2] = 6.0;
19     printf("Aluno: %s\n", aluno.nome);
20     printf("Notas: %.1f, %.1f, %.1f", aluno.nota[0], aluno.nota[1], aluno.nota[2]);
21     return 0;
22 }
```

Figura 87. Utilização de vetores como campos de estruturas

Agora, acessamos o membro da estrutura através do operador ponto (.) e, após, através dos colchetes acessamos a posição específica do vetor membro, como pode ser visto nas linhas 16, 17 e 18. O resultado da execução deste exemplo está na Figura 88.

Console

```
<terminated> Laboratorio.exe [C/C++ Applicatio
Aluno: Ricardo
Notas: 9.0, 8.5, 6.0
```

Figura 88. Resultado da execução

É possível mesclar as duas coisas, ou seja, criar um vetor de estruturas que contenham vetores como membros. Dessa forma, podemos também inicialiá-lo de forma prática, no momento da declaração, como vimos na unidade anterior. Segue, na Figura 89, um exemplo de utilização desta forma:

Laboratorio.c

```c
// Uso de vetores de estruturas
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct s_aluno tipoAluno;

struct s_aluno {
    char nome[50];
    float nota[3];
};

int main(void) {
    tipoAluno turma[2] = { {"Ricardo", {10.0, 8.5, 7.5} },
                           {"Nayane", {8.5, 6.0, 7.5} } };
    int i;
    for (i = 0; i < 2; i++) {
        float media = (turma[i].nota[0] + turma[i].nota[0] + turma[i].nota[0]) /3;
        printf("Aluno: %s\n", turma[i].nome);
        printf("Notas: %.1f, %.1f, %.1f\n", turma[i].nota[0],
                turma[i].nota[1], turma[i].nota[2]);
        printf("Média: %.1f\n", media);
    }
    return 0;
}
```

Figura 89. Exemplo de utilização de vetores com estruturas

Neste programa, vemos como se dá o acesso a posições de elementos do tipo vetor em vetores de estruturas. Cada chamada terá dois índices entre colchetes, o primeiro indicando a posição do vetor de estruturas. O segundo indicando a posição do vetor membro da estrutura. O resultado da execução deste programa encontra-se na Figura 90.

Console

```
<terminated> Laboratorio.exe [C/C++ Application] D:\
Aluno: Ricardo
Notas: 10.0, 8.5, 7.5
Média: 10.0
Aluno: Nayane
Notas: 8.5, 6.0, 7.5
Média: 8.5
```

Figura 90. Resultado da execução

Estruturas de Estruturas

Temos a possibilidade, ainda, de introduzir membros nas estruturas que são também estruturas. Por exemplo, se quisermos que a estrutura aluno tenha um membro data de nascimento, usando a estrutura mostrada no início desta unidade, podemos fazer este agrupamento da seguinte maneira (Figura 91):

Laboratorio.c

```
1  // Uso de estruturas de estruturas
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  typedef struct s_aluno tipoAluno;
7  typedef struct t_data tipoData;
8
9  struct t_data {
10     int dia;
11     int mes;
12     int ano;
13 };
14
15 struct s_aluno {
16     char nome[50];
17     float nota[3];
18     tipoData nascimento;
19 };
20
21 int main(void) {
22     tipoAluno aluno;
23     strcpy(aluno.nome, "Ricardo");
24     aluno.nota[0] = 9.8;
25     aluno.nota[1] = 8.5;
26     aluno.nota[2] = 7.2;
27     aluno.nascimento.dia = 18;
28     aluno.nascimento.mes = 03;
29     aluno.nascimento.ano = 1986;
30     return 0;
31 }
```

Figura 91. Uso de estruturas de estruturas

Para acessar os membros que são estruturas, também usamos o operador ponto (.) seguido do nome do membro, e para acessar os membros desses atributos, colocamos novamente o operador ponto (.) seguido, agora, do nome do membro na estrutura interna (linhas 24, 25 e 26). Dessa forma, a cada nível que “entrarmos” em nossa estrutura, colocaremos um novo ponto indicando membros de membros, tanto para leitura quanto para escrita de dados.

Exercícios Propostos

1. (Resolvido) Crie um programa de agenda telefônica, com os dados de nome do tipo string e telefone do tipo *string*, para cada contato, armazenados em um registro. O programa deverá criar um vetor de 10 posições e iniciá-lo no momento da declaração. Deverá, ainda, mostrar uma tabela de nomes e telefones com os dados armazenados no vetor.

Laboratorio.c

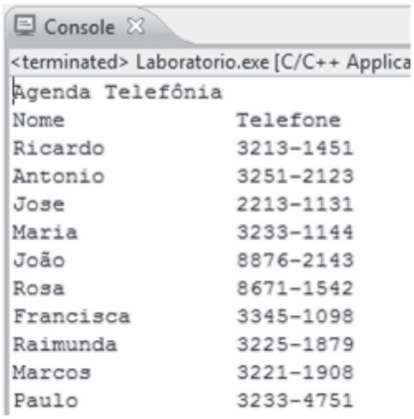
```
#include<stdio.h>
#include<stdlib.h>

typedef struct contato tipoContato;

struct contato {
    char nome[50];
    char telefone[10];
};

int main() {
    tipoContato agenda[10] = {
            { "Ricardo  ", "3213-1451" },
            { "Antonio  ", "3251-2123" },
            { "Jose     ", "2213-1131" },
            { "Maria    ", "3233-1144" },
            { "João     ", "8876-2143" },
            { "Rosa     ", "8671-1542" },
            { "Francisca", "3345-1098" },
            { "Raimunda ", "3225-1879" },
            { "Marcos   ", "3221-1908" },
            { "Paulo    ", "3233-4751" }
    };
    int i;
    printf ("Agenda Telefônia\n");
    printf ("Nome     \t Telefone\n");
    for (i = 0; i < 10; i++) {
        printf ("%s \t %s\n", agenda[i].nome, agenda[i].telefone);
    }
    return 0;
}
```

Execução:



2. Crie uma estrutura para representar os dados de um funcionário, com matrícula de tipo inteiro, nome de tipo string, estado civil de tipo string, endereço do tipo string, cargo de tipo string, salário de tipo float e data de nascimento de tipo data. O tipo data deve ser definido como uma estrutura com os membros dia, mês e ano de tipo inteiro. Com essa estrutura faça as seguintes operações:

a. Crie um vetor de 20 posições para armazenar o cadastro de funcionários de uma empresa fictícia;

b. Crie uma variável fim que demarcará a última posição atualmente preenchida em seu cadastro de funcionários;

c. Faça com que o programa apresente um menu onde o usuário possa escolher entre as funções de cadastrar funcionário, excluir funcionário, alterar funcionário, buscar funcionário, listar todos os funcionários e sair.

d. O usuário deverá poder escolher qualquer das opções, qualquer quantidade de vezes. Somente ao escolher a opção sair o programa deve ser finalizado;

e. Quando a opção cadastrar funcionário for escolhida, leia todos os dados de um funcionário e o inclua na primeira posição livre do vetor. A variável fim deve ser aumentada de um.

f. Se já houver vinte funcionários cadastrados (limite do nosso vetor), o programa deverá mostrar a mensagem “Impossível cadastrar. Vetor cheio”;

g. Quando a opção excluir funcionário for escolhida, o programa deverá solicitar a posição que o usuário deseja excluir. Caso a opção esteja preenchida com um funcionário no vetor, todos os elementos, a partir daquela posição para frente devem ser “puxados” uma posição para trás, para refletir a exclusão. A variável fim deve ser

reduzida de um. Caso contrário, deverá ser exibida a mensagem “Impossível excluir. Posição inválida”;

h. Quando a opção alterar funcionário for escolhida, o usuário deverá escolher a posição a alterar. Caso esta posição esteja preenchida com os dados de um funcionário, o programa deverá mostrar os dados atuais e pedir para o usuário digitar novamente todos os dados para aquele funcionário. Caso contrário, deverá mostrar a mensagem “Impossível alterar. Posição não preenchida”;

i. Quando a opção buscar funcionário for escolhida, o programa deverá solicitar ao usuário o nome a procurar. Após isso, o programa irá buscar, entre os funcionários cadastrados, um com nome procurado (ou parte dele). Caso encontre, deverá mostrar todos os dados desse funcionário. Caso contrário, deverá exibir a mensagem “Funcionário não encontrado”;

j. Quando a opção listar todos os funcionários for selecionada, o programa deve exibir todos os dados de todos os funcionários cadastrados até o momento. Se nenhum funcionário estiver cadastrado, deverá exibir a mensagem “Nenhum funcionário cadastrado”;

l. Quando a opção sair for escolhida, o programa deverá finalizar.



Resumindo

Na unidade três estudamos vetores e matrizes, construções homogêneas da linguagem C, ou seja, só podíamos armazenar valores do mesmo tipo. Nessa unidade introduzimos o conceito de estruturas, através das quais armazenamos dados de tipos diferentes na mesma construção. Vimos que, aliando estruturas ao uso de vetores ou matrizes, podemos alçar nossos programas a um nível bem mais complexo e com manipulação de dados mais simples, na mesma construção, ao invés de criarmos vários vetores.

UNIDADE 05

Funções e Procedimentos

5 FUNÇÕES E PROCEDIMENTOS

Funções

Assim como na matemática, funções são usadas em programação para aplicarmos a determinadas entradas e obtermos um valor resultante. Primeiramente, veremos a declaração de funções simples, sem parâmetros, para depois passarmos a noção de passagem de parâmetros e às formas de se fazer isso.

Usando funções criamos pequenos pedaços de códigos separados do programa principal. Na verdade, tudo que escrevemos em C são funções. A própria *main* é uma função, como estamos chamando desde o início dos estudos. Tendo o nome de principal, ela é a função mais importante do programa, pois é a partir dela que nosso programa iniciará sua execução. Sendo assim, sua existência no programa é obrigatória.

Analisemos um exemplo que possui somente uma função *main* (Figura 92) e que não faça nada para verificarmos o conceito e o modo de declaração de função.

Laboratorio.c

```
1 // Função main
2 int main() {
3     return 0;
4 }
```

Figura 92. Exemplo de função main

Este é o programa mais simples que podemos fazer na linguagem C. Temos apenas a função *main* declarada e, dentro do bloco da função, os comandos que ela irá executar. Vamos detalhar essa declaração mostrando a forma geral de declaração de funções em C:

```
tipo_de_retorno nome_da_função ( ) {
comandos;
}
```

A primeira coisa a ser feita é especificar qual tipo a função irá retornar. Assim como funções matemáticas que calculam um valor, as funções em C também o fazem, podendo ser retornado qualquer tipo dos que foram vistos anteriormente, inclusive vetores e estruturas. Em nossa função *main* o tipo de retorno padrão é *int* (linha 2).

Logo após colocarmos o nome da função, através desse nome a função pode ser chamada de outras partes do programa, como veremos posteriormente. Para a especificação do nome, valem as mesmas regras que vimos para nomenclatura de variáveis. O nome *main* é um nome de função especial, que indicará que se trata da função principal do programa, aquela que será chamada no momento do início da execução (linha 2).

Em seguida abrimos e fechamos parênteses. Entre esses parêntesis ficarão os parâmetros da função, só que, no momento, não abordaremos isso, ficando mais para frente. Feito isso a declaração estará finalizada (linha 2), podemos abrir o bloco de comandos da função com { para começar a escrever os comandos que a função terá. Após o último comando da função devemos fechá-la com }.

Toda função deve retornar um valor do tipo que foi especificado em sua declaração. Isso é feito através do comando *return*, através dele especificamos que valor vai ser retornado. Dessa forma, o último comando em toda função é sempre um *return*. No caso da função *main,* que sempre retorna inteiro, o valor padrão de retorno é 0, como podemos ver na linha 3.

Com exceção da função *main*, todas as outras funções são secundárias, o que significa que elas podem existir ou não. As grandes vantagens de usarmos funções em nossos programas são porque elas retornam valores, ajudam a fragmentar o código em partes menores e podem ser utilizadas mais de uma vez no mesmo programa.

Criando funções

Primeiramente, veremos as funções sem parâmetros em C. Uma função é como uma variável que guarda várias linhas de código. Sendo assim, para que possamos chamá-las em determinada parte do código, é

preciso que primeiro tenhamos declarado as mesmas. Sendo assim, toda função deve ser declarada antes da função *main*.

Para chamarmos uma função, basta evocarmos o nome que usamos para criá-la, seguido da lista de parâmetros entre parênteses. Como, em nosso caso, ainda não teremos parâmetros, basta colocar os parênteses vazios. Vejamos o exemplo a seguir na **Figura 93**:

Laboratorio.c

```
1  // Declaração e uso de função
2  #include<stdio.h>
3  #include<stdlib.h>
4
5  int dez() {
6      printf("Ola, eu sou uma função\n");
7      printf("Retorno o valor 10\n");
8      return 10;
9  }
10
11 int main() {
12     int x = dez();
13     printf("%d", x);
14     return 0;
15 }
```

Figura 93. Declaração e uso de uma função

Temos uma função chamada dez (linha 5) que deve retornar algum valor do tipo inteiro. Devido a isso, na função *main*, atribuiremos esse resultado a uma variável do mesmo tipo (linha 13) para capturarmos o valor retornado pela função. A partir disso, essa variável assumirá o valor 10 e poderemos usá-la no decorrer do programa.

Como resultado da execução desse programa, a função será chamada, serão impressos os dois textos das linhas 6 e 7 e, por fim, será retornado o valor 10 pela função. Na linha 13, o valor da variável x, que, nesse caso, é o valor retornado pela função, será impresso. Podemos ver esse resultado da execução na Figura 94.

Console

```
<terminated> Laboratorio.exe [C/C++ Applicatio
Ola, eu sou uma função
Retorno o valor 10
10
```

Figura 94. Resultado da execução

Na Figura 95 temos outro exemplo de função que calculará a soma de dois valores e retornará esse valor como resultado da função.

Laboratorio.c

```
1  // Função soma
2  #include<stdio.h>
3  #include<stdlib.h>
4
5  int soma() {
6      int a, b;
7      printf("Digite um número: ");
8      fflush(stdout);
9      scanf("%d", &a);
10     printf("Digite outro número: ");
11     fflush(stdout);
12     scanf("%d", &b);
13     return a + b;
14 }
15
16 int main() {
17     printf("Soma: %d", soma());
18     return 0;
19 }
```

Figura 95. Função que calcula a soma

Nesse exemplo, dentro da própria função soma, pedimos para o usuário digitar dois valores (linhas 7 a 12) e, ao final, retornamos a soma dos dois valores lidos (linha 13). Observamos que podemos usar qualquer tipo de expressão na chamada *return*, contanto que o resultado dessa expressão seja do tipo de retorno da função (nesse caso, int).

É importante percebermos também que a chamada da função não necessita ser um comando isolado. Nesse exemplo usamos a função soma direto na chamada da função *printf* (linha 17), pois apenas queremos imprimir o resultado e não armazenar esse valor numa variável para ser usada depois.

Procedimentos

Um procedimento tem a mesma definição de uma função e, consequentemente, a mesma forma de declaração. Também representa um bloco de código funcional que pode ser chamado a partir de qualquer lugar do programa.

A diferença está no fato de que um procedimento não retorna valor algum. Por causa disso, em um procedimento não teremos o comando *return*. E para indicarmos que será um procedimento e não uma função, usamos um tipo especial que dirá ao programa que a função não tem retorno. Esse tipo é o *void*.

Sendo assim, o formato de declaração padrão de um procedimento é o seguinte:

```
void nome_do_procedimento ( ) {
}
```

Usamos procedimentos quando queremos executar um conjunto de linhas de código em várias partes distintas de nosso programa, mas não queremos retornar nenhum valor específico. Vejamos o exemplo da Figura 96.

Laboratorio.c

```
1  // Declaração e uso de procediemnto
2  #include<stdio.h>
3  #include<stdlib.h>
4
5  void mensagem() {
6      printf("Olá mundo\n");
7      printf("Sou um procedimento");
8  }
9
10 int main() {
11     mensagem();
12     return 0;
13 }
```

Figura 96. Exemplo de procedimento

O procedimento mensagem, declarado na linha 5, tem apenas dois comandos de escrita (printf nas linhas 6 e 7), ou seja, não precisa retornar valor algum. A chamada a esse procedimento é feita da mesma forma que as funções (linha 11), através do nome do procedimento seguido dos parâmetros entre parênteses (como não há nenhum ainda, ficam somente os parênteses). O resultado da execução deste programa está na Figura 97.

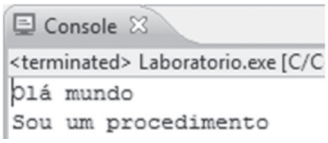


Figura 97. Resultado da execução

As funções e os procedimentos têm utilizações diferentes em nossos programas. Em resumo, podemos dizer que tudo são funções, mesmo os procedimentos podem ser definidos como funções que não retornam nada. Ficará a cargo do programador escolher qual dos dois tipos se enquadra no problema que está querendo resolver.

Passagem de parâmetros

As funções que vimos até agora não precisavam de nenhum valor de entrada, ou seja, não tinham parâmetros. Se quisermos passar algum valor como entrada para a função (ou procedimento), para ser usado em seus cálculos internos, devemos especificá-los entre os parênteses após o nome da função, da seguinte forma:

```
tipo_de_retorno nome_da_função (tipo1 nome1, ... , tipon nomen ) {
comandos;
}
```

Podemos declarar quantos parâmetros quisermos, todos com seu próprio tipo e separados por vírgula. Dessa forma, podemos reescrever a função soma para que não precisemos, dentro da função apenas, pedir para o usuário digitar os parâmetros. Vejamos na **Figura 98**:

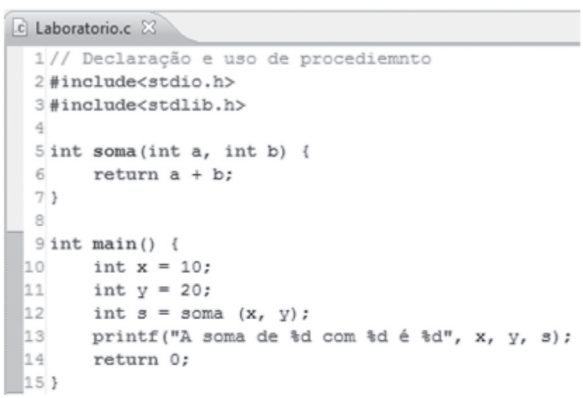


Figura 98. Declaração de função com parâmetros

Vejamos que, nesse caso, nossa função está bem mais compacta. Esta é a grande vantagem de programarmos utilizando funções. Cada item que quisermos implementar, em separado, pensamos em funções de forma a simplificar a programação e facilitar o entendimento. O resultado da execução desse programa pode ser visto na **Figura 99**:

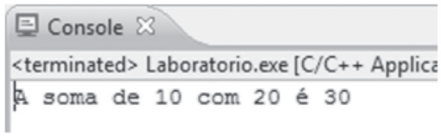


Figura 99. Resultado da execução

Vale ressaltar, ainda, que podemos ter várias funções com o mesmo nome, mas com parâmetros diferentes. A distinção no momento da chamada é feita de acordo com os parâmetros passados.

Exercícios Propostos

1. Faça um programa que contenha uma função subtração, que receba dois valores e retorne a subtração dos dois.
2. Faça um programa que contenha uma função que receba um nome e escreva “Bom dia” para esse nome recebido.
3. Faça um programa que contenha duas funções que recebam o raio de um círculo e retornem: uma a área da circunferência e a outra o comprimento da mesma.
4. Faça um programa que contenha quatro funções: soma, subtração, multiplicação e divisão. Seu programa deverá proporcionar ao usuário, através de uma função menu, a escolha de qual das operações deseja executar.
5. (Resolvido) Faça um programa que contenha uma função receba uma temperatura em graus Celcius e retorne sua correspondente transformação para Farenheit.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>

float celciusToFarenheit(float c) {
    return c * 1.8 + 32;
}

int main() {
    float temp = 0;
    temp = celciusToFarenheit(temp);
    printf("%.2f\n", temp);
    temp = 40;
    temp = celciusToFarenheit(temp);
    printf("%.2f\n", temp);
    return 0;
}
```

Execução:

Console

```
<terminated> Laboratorio.exe
32.00
104.00
```

6. Faça um programa que contenha um procedimento que receba um vetor e imprima todos os seus valores. Para que uma função receba um vetor como parâmetro, declaramos da seguinte forma: *void* escreve (int[10] vetor), por exemplo. Se não quisermos especificar o tamanho do vetor, podemos fazer da seguinte forma: *void* escreve (int[] vetor), no entanto, devemos informar a quantidade de posições de alguma forma. A maneira mais simples é através de outro parâmetro: *void* escreve (int[] vetor, int tamanho).

7. Faça um programa que contenha uma função que receba dois vetores e retorne o vetor resultante da soma desses dois. Para que uma função retorne um vetor, devemos declará-la com um asterisco logo antes do nome, por exemplo: int *soma(int[] vetor1, int tam1, int[] vetor2, int tam2).

8. Faça um programa que contenha uma função que receba os valores da base e altura de um retângulo e retorne sua área.

9. Faça um programa que contenha uma estrutura pessoa com os seguintes membros: nome (string), endereço (string), telefone (string) e data de nascimento (tipo data, com membros dia, mês e ano). Seu

programa deverá ter funções (ou procedimentos) que executem as seguintes tarefas:

a. Receber todos os dados de uma pessoa;
b. Escrever na tela todos os dados de uma pessoa;
c. Apagar todos os dados lidos da pessoa (ou seja, inserir vazios nos valores *string* e zeros nos valores numéricos);
d. Verificar se a pessoa com os dados colocados é maior de idade (mais de 18 anos);
e. Faça um menu para que o usuário possa escolher qual das operações quer escolher.

10. Faça um programa que contenha uma função que receba três valores e retorne o menor entre eles.
11. Faça um programa que contenha uma função que receba uma *string* e retorne quantas consoantes existem na *string* recebida.
12. Faça um programa que contenha uma função que receba uma *string* e retorne outra *string* com todas as letras substituídas pela correspondente maiúscula.
13. Faça um programa que contenha uma função que receba um valor inteiro e retorne o número com seus dígitos invertidos. Por exemplo, dado o número 4892, a função deve retornar o valor 2984.
14. Faça um programa que contenha duas funções que recebam dois inteiros e retornem: uma o mdc e outra o mmc entre os números recebidos.
15. Faça um programa que contenha uma função que receba um número e verifique se ele é primo ou não.

Variáveis locais e globais

Quando começamos a usar funções e procedimentos em nossos programas, devemos entender dois novos conceitos que nos ajudarão no desenvolvimento de nossos programas: variáveis locais e variáveis globais.

Variáveis locais

Variáveis locais são variáveis declaradas dentro de funções específicas. Por causa disso, essas variáveis são visíveis apenas dentro daquela função, ou seja, só podem ser usadas na própria função. Variáveis

locais não podem ser usadas em outro lugar do programa. Vejamos o exemplo da Figura 100:

Laboratorio.c

```
1  // Uso de variáveis locais
2  #include<stdio.h>
3  #include<stdlib.h>
4
5  void imprime(int a[], int tam) {
6      int i;  // Local a imprime
7      for (i = 0; i < tam; i ++) {
8          printf("%d, ", a[i]);
9      }
10 }
11
12 int main() {
13     int x[10] = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
14     imprime(x, 10);
15     return 0;
16 }
```

Figura 100. Variáveis locais

A variável i, declarada dentro do procedimento imprime (linha 6), só pode ser usada dentro deste procedimento. Ou seja, o programador só pode atribuir valores a ela ou usar o valor nela contido enquanto estiver dentro do procedimento imprime. Da mesma forma, o vetor x, declarado na linha 13, só pode ser usado na função *main*.

O resultado da execução deste programa pode ser visto na Figura 101:

Console

```
<terminated> Laboratorio.exe [C/C++ Application] D:\cpp
2, 4, 6, 8, 10, 12, 14, 16, 18, 20,
```

Figura 101. Resultado da execução

Variáveis globais

Variáveis globais são visíveis em qualquer ponto do programa. São declaradas fora de qualquer função e, normalmente, logo no início do programa, pois, a única limitação, nesse caso, é que só pode ser usada após ser declarada. Podem ser usadas dentro de qualquer função e, por consequência, dentro da *main*. Vejamos o mesmo exemplo anterior, agora, usando variáveis globais (Figura 102):

Laboratorio.c

```
1  // Uso de variáveis globais
2  #include<stdio.h>
3  #include<stdlib.h>
4
5  int x[10] = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
6
7  void imprime(int tam) {
8      int i;  // Local a imprime
9      for (i = 0; i < tam; i ++) {
10         printf("%d, ", x[i]);
11     }
12 }
13
14 int main() {
15     imprime(10);
16     return 0;
17 }
```

Figura 102. Uso de variáveis globais

A variável i, declarada dentro da função imprime, continua sendo variável local ao procedimento imprime. A variável x foi levada para fora da função *main*, antes da função imprime, sendo assim, é uma variável global e podemos usá-la diretamente nas funções imprime e *main*, como fizemos na linha 10.

Isso evita, ainda, que tenhamos que passar o vetor x como parâmetro (linha 15), já que temos acesso a ele nas duas funções. O resultado da execução deste programa é o mesmo já visto na Figura 102.

No entanto, devemos ficar atentos ao usar variáveis globais. Ser variável global significa estar na memória durante toda a execução do programa, sendo assim, qualquer alteração na variável modificará verdadeiramente a variável. Vejamos o exemplo a seguir (Figura 103):

Laboratorio.c

```
1  // Uso de variáveis globais
2  #include<stdio.h>
3  #include<stdlib.h>
4
5  int x = 8;
6
7  void imprime(int y) {
8      x = 10;
9      y = 20;
10     printf("%d %d\n", x, y);
11 }
12
13 int main() {
14     int a = 2;
15     printf("%d %d\n", x, a);
16     imprime(a);
17     printf("%d %d\n", x, a);
18     return 0;
19 }
```

Figura 103. Análise de variáveis globais

Vejamos o que acontece nesse exemplo. Estamos usando duas variáveis: a global x e a local a. Na linha 15 imprimimos o conteúdo inicial das duas variáveis, na linha 16, chamamos a função imprime, passando a variável a como parâmetro. Na função imprime, alteramos o conteúdo das duas variáveis nas linhas 8 e 9 e imprimimos os respectivos conteúdos (linha 10). Na linha 17, ao final do programa, imprimimos novamente os conteúdos dessas variáveis. Vejamos, na Figura 104, o resultado da execução desse programa:

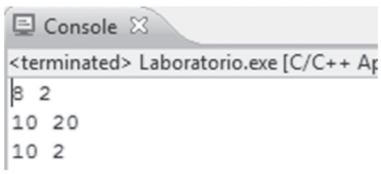


Figura 104. Resultado da execução

É importante perceber que, no primeiro caso, os valores iniciais 8 e 2 são impressos. No segundo passo, dentro da função imprime, os valores alterados das funções são mostrados. No entanto, ao final, são impressos novamente os valores e a variável a volta a ter seu conteúdo inicial. Isso acontece porque, dentro do procedimento, a variável a é representada pelo parâmetro y, uma variável local. Dessa forma, o conteúdo de y não se mantém fora do procedimento, diferente de x, por ser uma variável global.

Devemos sempre evitar o uso de variáveis globais, pois diminui a legibilidade do código e pode causar problemas de alterações de variáveis não previstas. Também diminui a independência de nossas funções com relação a outros códigos. Se quisermos refletir esse comportamento que acabamos de analisar devemos usar o mecanismo de passagem de parâmetros por referência.

Passagem de parâmetros por valor e por referência

Os valores que colocamos como entrada de nossas funções podem ser passadas de duas formas: por valor ou por referência. Vamos analisar cada uma delas.

Passagem de parâmetros por valor

Passar um parâmetro por valor significa que somente o valor da variável será copiado para dentro da função (ou procedimento). Usamos esta forma de passagem da forma como já vínhamos fazendo desde o início, apenas declarando os parâmetros com nome e tipo. Sendo assim, os exemplos vistos anteriormente usam passagem de parâmetros por valor. Vejamos mais um exemplo na Figura 105:

Laboratorio.c

```
1  // Passagem de parametros por valor
2  #include<stdio.h>
3  #include<stdlib.h>
4
5  void imprime(int numero) {
6      printf("%d \n", numero);
7      numero = 50;
8      printf("%d \n", numero);
9  }
10
11 int main() {
12     int n = 2;
13     printf("%d \n", n);
14     imprime(n);
15     printf("%d \n", n);
16     return 0;
17 }
```

Figura 105. Passagem de parâmetros por valor

Neste exemplo passamos a variável n para a função imprime usando passagem por valor. Sendo assim, apesar de alterarmos o valor do parâmetro número na linha 7, essa alteração não será refletida fora do programa. Por isso, quando imprimimos novamente a variável n na linha 15, seu valor será o original, ou seja, 2.

Console

```
<terminated> Laboratorio
2
2
50
2
```

Figura 106. Resultado da execução

Passagem de parâmetros por referência

A passagem por referência faz o contrário da passagem por valor. Ou seja, as modificações feitas dentro de uma função são refletidas na variável passada como parâmetro. E, fora da função, o novo valor obtido dentro da função poderá ser utilizado.

Para passarmos por valor usamos a notação * antes do nome da variável que estamos querendo, junto ao nome, da seguinte forma:

```
tipo nome_da_função (tipo *parâmetro_passado_por_referencia) {
}
```

Na verdade, fazendo isso estamos passando uma referência para o local de memória onde está realmente armazenada a variável passada, ou seja, um ponteiro. É assim que se torna possível simular a passagem por referência: passamos o endereço de memória e a função altera quem estiver naquele endereço, no caso, uma variável declarada fora da função. Com isso, também devemos colocar o * antes do nome da variável sempre que quisermos usar essa variável dentro da função. Vejamos o exemplo da Figura 107:

Laboratorio.c

```
1  // Passagem de parametros por referência
2  #include<stdio.h>
3  #include<stdlib.h>
4
5  void imprime(int *numero) {
6      printf("%d \n", *numero);
7      *numero = 50;
8      printf("%d \n", *numero);
9  }
10
11 int main() {
12     int n = 2;
13     printf("%d \n", n);
14     imprime(&n);
15     printf("%d \n", n);
16     return 0;
17 }
```

Figura 107. Passagem de parâmetros por referência

Nesse caso, a variável n realmente será alterada dentro da função imprime. Após a chamada desta função na linha 14, n irá adquirir um novo valor na atribuição da linha 7. Seu novo valor deverá ser impresso pelo *printf* da linha 15 (50).

Devemos perceber ainda que como o parâmetro é um ponteiro

e não um inteiro, propriamente dito, devemos passar um endereço de memória. Para isso, usamos o operador &, da mesma forma que usamos no *scanf*, para indicar o endereço de memória da variável e não somente a variável.

O resultado da execução deste programa pode ser visto na Figura 108:

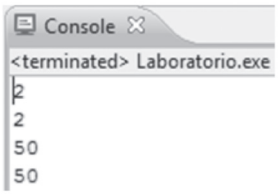


Figura 108. Resultado da execução

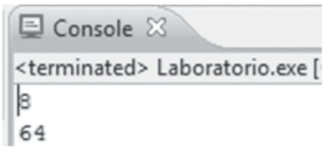
Exercícios Propostos

1. Faça um programa que contenha uma função que receba dois parâmetros inteiros por referência e troque o valor dessas variáveis.
2. (Resolvido) Faça um programa que contenha uma função que receba dois números e calcule a potência do primeiro em relação ao segundo, retornando o valor através da passagem por referência do primeiro parâmetro.

Laboratorio.c

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 void potencia(int*, int);
5
6 int main() {
7     int a = 2, b = 3;
8     potencia(&a, b);
9     printf("%d\n", a);
10    a = 4;
11    potencia(&a, b);
12    printf("%d\n", a);
13    return 0;
14 }
15
16 void potencia(int *x, int y) {
17    int i, aux = *x;
18    for (i = 1; i < y; i++) {
19        *x = *x * aux;
20    }
21 }
```

Execução:



3. Faça um programa que contenha uma função que receba três parâmetros por referência e os ordene, alterando para ser o primeiro parâmetro o menor, o segundo parâmetro o intermediário e o terceiro parâmetro o maior.
4. Faça um programa que contenha uma função que receba um valor inteiro como referência e retorne o resto da divisão deste número por 10. Altere também o valor da variável passada por referência, dividindo-a por 10.
5. Faça um programa que contenha uma função que receba uma *string* por referência e altere essa *string* para seu valor ao contrário.
6. Faça um programa que contenha uma função que receba um número i e retorne o i-ésimo número primo existente nesse mesmo parâmetro passado por referência.
7. Faça um programa que contenha uma função, receba um parâmetro (inteiro, por valor) com o total de minutos passados ao longo do dia e receba também dois parâmetros (inteiros, por referência) no qual deve preencher com o valor da hora e do minuto corrente. Seu programa deverá ler do teclado quantos minutos se passaram desde meia-noite e imprimir a hora corrente (use a sua função).

Protótipo de funções

Conforme explicado anteriormente, todas as funções precisam ser declaradas antes de serem utilizadas. No entanto, normalmente, a primeira função que implementamos é a própria *main*, de forma a mantermos nosso código organizado e identificarmos facilmente qual a funcionalidade principal do nosso programa. Somente após a *main* virão nossas funções personalizadas.

No entanto, precisamos usar as funções que implementaremos,

a partir de agora, após a função *main*. Para que nossas funções sejam reconhecidas dentro da *main*, declaramos apenas o protótipo das funções antes da função principal. Dessa forma, elas serão reconhecidas e podemos usá-las na principal, mesmo sem termos implementado ainda.

Para declarar o protótipo de uma função, basta escrevermos o cabeçalho da função com os mesmos parâmetros que ela terá, seguida de ponto e vírgula. Colocamos somente essa assinatura antes da *main*, e depois dela implementamos a função propriamente dita. Vejamos exemplo usado anteriormente com a declaração do protótipo (Figura 109):

Laboratorio.c

```
1  // Protótipo de funções
2  #include<stdio.h>
3  #include<stdlib.h>
4
5  void imprime(int *numero);
6
7  int main() {
8      int n = 2;
9      printf("%d \n", n);
10     imprime(&n);
11     printf("%d \n", n);
12     return 0;
13 }
14
15 void imprime(int *numero) {
16     printf("%d \n", *numero);
17     *numero = 50;
18     printf("%d \n", *numero);
19 }
```

Figura 109. Protótipo de funções

Dessa forma, a função imprime, cujo protótipo está descrito na linha 5, será reconhecida e, por conseguinte, pode ser usada mesmo sem ter sido implementada antes da função *main*. O resultado da execução deste programa é o mesmo visto na Figura 109.

Vale ressaltar ainda que, no protótipo da função, podemos suprimir os nomes dos parâmetros, deixando para especificá-los apenas no momento da declaração. Na figura 110 temos um exemplo dessa utilização:

```c
// Protótipo de funções
#include<stdio.h>
#include<stdlib.h>

void troca(int*, int*);

int main() {
    int x = 20;
    int y = 40;
    printf("%d %d\n", x, y);
    troca(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}

void troca(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Figura 110. Supressão de nomes de parâmetros no protótipo

O resultado da execução deste programa encontra-se na Figura 111:

```
<terminated> Laboratorio.exe
20 40
40 20
```

Figura 111. Resultado da execução

Usando a prototipagem simplificamos nosso código. Teremos uma parte destinada a declarar as funções (protótipos) e logo após temos a função principal que é o programa. Logo abaixo da função principal estão as funções secundárias implementadas.

Recursividade

Recursividade não é um comando propriamente dito e sim uma técnica ou “habilidade” que diversas linguagens proporcionam. Recursão é quando uma função chama a si própria ou chama outra função e esta chama a primeira.

Para usar a recursividade basta chamarmos uma função, como já vínhamos fazendo, só que agora dentro do bloco de comandos da própria função que está sendo definida. Essa característica é especialmente útil quando a função que estamos implementando é definida em termos dela mesma.

Vamos pensar na função fatorial: como sabemos, o fatorial de um número N é o valor N multiplicado pelo fatorial de N-1. Ou seja, a função fatorial é eminentemente recursiva. Vejamos como ficaria sua definição na linguagem de programação C (Figura 112):

Laboratorio.c

```
1  // Protótipo de funções
2  #include<stdio.h>
3  #include<stdlib.h>
4
5  int fatorial(int);
6
7  int main() {
8      printf("%d\n", fatorial(5));
9      return 0;
10 }
11
12 int fatorial(int n) {
13     if (n == 1) {
14         return 1;
15     } else {
16         return n * fatorial(n - 1);
17     }
18 }
```

Figura 112. Implementação recursiva do fatorial

Vejamos que, na linha 16, dentro da própria função fatorial fazemos uma chamada à função fatorial. Dessa forma, definimos a função exatamente da forma como ela é na realidade.

Devemos ter o cuidado de sempre que usarmos recursividade colocarmos uma condição de parada, ou seja, uma condição que terminará com as chamadas repetidas. Se isso não acontecer, nosso programa entrará em um *loop* de profundidade e nunca finalizará sua execução. No caso do fatorial, a condição de para é n = 1, pois para ele o resultado do fatorial é conhecido e igual a 1 (linha 13).

As chamadas à função fatorial estão descritas logo a seguir:

fatorial(5) =
5 * fatorial(4) =

5 * 4 * fatorial(3) =
5 * 4 * 3 * fatorial(2) =
5 * 4 * 3 * 2 * fatorial(1) =
5 * 4 * 3 * 2 * 1 =
5 * 4 * 3 * 2 =
5 * 4 * 6 =
5 * 24 = 120

Vejamos que somente ao chegar à condição de parada (n = 1) a recursão volta e calcula o resultado das multiplicações para obter o resultado final. O resultado da execução deste programa pode ser conferido na Figura 113.

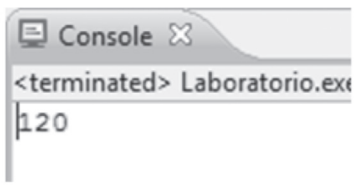


Figura 113. Resultado da execução

No entanto, esta facilidade tem seu custo. Apesar de simplificar nossos algoritmos, o uso da recursividade torna a execução do nosso programa mais custosa, visto que são efetuadas várias chamadas de função. Uma chamada de função leva mais tempo para ser processada que um comando de repetição comum.

Portanto, a recursividade deve ser usada com cautela. Às vezes, um algoritmo um pouco mais complexo que usa um pouco mais de variáveis pode ser necessário para que se tenha um desempenho melhor.



Exercícios Propostos

1. (Resolvido) Faça um programa que contenha uma função recursiva que receba dois números e calcule o mdc (Máximo Divisor Comum) entre eles.

Laboratorio.c

```c
#include<stdio.h>
#include<stdlib.h>

int mdc(int, int);

int main() {
    printf("%d\n", mdc(10, 4));
    printf("%d\n", mdc(10, 15));
    printf("%d\n", mdc(6, 24));
    printf("%d\n", mdc(336, 156));
    return 0;
}

int mdc(int x, int y) {
    int maior = x, menor = y;
    if (x < y) {
        maior = x;
        menor = y;
    }
    int resto = maior % menor;
    if (resto == 0) {
        return menor;
    } else {
        return mdc(menor, resto);
    }
}
```

Execução:

Console

<terminated> Laboratorio.exe

```
2
5
6
12
```

2. Faça um programa que contenha uma função recursiva, receba um número i e retorne o i-ésimo termo da série de Fibonacci. Use-o para fazer receber o número i do usuário e escrever a série até esse i-ésimo termo. A série de Fibonacci tem seus números obtidos a partir da soma dos dois valores anteriores, sendo que os dois primeiros são um: 1 1 2 3 5 8 13.

Ex: Entrada: 5. Saída: 1 1 2 3 5

3. Implemente, através de funções recursivas, o algoritmo conhecido como torres de Hanoi. Nesse problema você tem n pinos na torre A, cada um de um tamanho, e deve movê-los para a torre B com a ajuda de uma torre auxiliar C. A única restrição é que um pino maior não deve ficar sobre um menor. Dica: esse problema é parecido com o fatorial, para resolver o problema grande devemos resolver o problema imediatamente menor (n - 1) e assim por diante até chegarmos ao problema mais simples, onde

há somente um pino em A para movê-lo para B. Veja o esquema:

hanoi(n, A, B) =
hanoi(n-1, A, C)
passa o pino grande de A para B
hanoi(n-1, C, B)



4. Trabalho Final: Crie uma estrutura para representar os dados de um aluno, com matrícula de tipo inteiro, nome de tipo *string*, endereço do tipo *string*, data de nascimento de tipo data e um vetor de disciplinas do tipo disciplina. O tipo data deve ser definido como uma estrutura com os membros dia, mês e ano de tipo inteiro. O tipo disciplina também deve ser definido como uma estrutura com nome do tipo *string*, professor do tipo *string* e notas como sendo um vetor de três *floats* para armazenar as notas do aluno naquela disciplina. Com essa estrutura faça as seguintes operações:

a. Crie um vetor de vinte posições para armazenar o cadastro de alunos da Universidade Aberta do Piauí.

b. Crie uma variável fim que demarcará a última posição atualmente preenchida em seu cadastro de alunos.

c. Preencha os dados de cinco alunos automaticamente via comandos, para que seja possível testar todas as funções mesmo sem cadastrar manualmente alguns alunos.

d. Faça com que o programa apresente um menu através de uma função onde o usuário possa escolher entre as funções de cadastrar aluno, excluir aluno, alterar aluno, buscar aluno, listar todos os alunos, listar aluno de determinada disciplina, listar alunos aprovados e sair. Cada funcionalidade deve ser feita como uma função em separado.

e. O usuário deverá poder escolher qualquer das opções, qualquer quantidade de vezes. Somente ao escolher a opção sair o programa deve ser finalizado.

f. Quando a opção cadastrar aluno for escolhida, leia todos os dados de um aluno e o inclua na primeira posição livre do vetor. A variável fim deve ser aumentada de um

g. Se já houver vinte alunos cadastrados (limite do nosso vetor), o programa deverá mostrar a mensagem “Impossível cadastrar. Vetor cheio”.

h. Quando a opção excluir aluno for escolhida, o programa deverá solicitar a posição que o usuário deseja excluir. Caso a opção esteja preenchida com um aluno no vetor, todos os elementos, a partir daquela posição para frente devem ser “puxados” uma posição para trás, para refletir a exclusão. A variável fim deve ser reduzida de um. Caso contrário, deverá ser exibida a mensagem “Impossível excluir. Posição inválida”.

i. Quando a opção alterar aluno for escolhida, o usuário deverá escolher a posição a alterar. Caso esta posição esteja preenchida com os dados de um aluno, o programa deverá mostrar os dados atuais e pedir para o usuário digitar novamente todos os dados para aquele aluno. Caso contrário, deverá mostrar a mensagem “Impossível alterar. Posição não preenchida”.

j. Quando a opção buscar aluno for escolhida, o programa deverá solicitar ao usuário o nome a procurar. Após isso, o programa irá buscar, entre os aluno cadastrados, um com nome procurado (ou parte dele). Caso encontre, deverá mostrar todos os dados desse aluno. Caso contrário, deverá exibir a mensagem “Aluno não encontrado”.

l. Quando a opção listar todos os alunos for selecionada, o programa deve exibir todos os dados de todos os alunos cadastrados até o momento. Se nenhum funcionário estiver cadastrado, deverá exibir a mensagem “Nenhum funcionário cadastrado”.

m. Quando a opção listar alunos de determinada disciplina for selecionada, o programa deve solicitar ao usuário o nome de uma disciplina e procurar e mostrar, entre os alunos cadastrados, os que estejam naquela disciplina. Liste todos os alunos com as respectivas notas, médias e resultados finais (AM para média >= 7, EF para 7 > média >= 4 e RN para média < 4). Caso não encontre nenhum, mostrar a mensagem “Nenhum aluno encontrado para a disciplina procurada”.

n. Quando a opção listar alunos aprovados for selecionada, o programa deverá procurar os alunos cadastrados que possuem

como média (soma das três notas na disciplina dividida por três) um valor maior ou igual a sete. Caso não seja encontrado nenhum aluno, mostrar a seguinte mensagem: “Nenhum aluno aprovado encontrado”.

I. Quando a opção sair for escolhida, o programa deverá finalizar.



Resumindo

Nessa última unidade estudamos os conceitos de funções e procedimentos, como forma de modular nossos programas, ou seja, como transformar problemas grandes em problemas menores e mais fáceis de resolver, de lidar e de usar. Dessa forma, damos mais clareza a nossos programas e reduzimos a quantidade de linhas de código por função. Assim, os nossos programas ficam mais fáceis de entender, de manipular e de alterar futuramente, caso necessário.

Referências

Exercícios de Introdução a algoritmos. Disponível por www em: http://www.guj.com.br/posts/list/120244.java, acesso em 10 de agosto de 2010, às 22:00.

ASCENCIO, Ana Fernanda Gomes; CAMPOS, Edilene A. V.i de. **Fundamentos da Programação de Computadores algoritmos**: Pascal e C/C++. 1ª ed. São Paulo: Pearson Education, 2003.

C++ Language Tutorial. Disponível por www em: http://www.cplusplus.com/doc/tutorial/, acesso em 10 de agosto de 2010, às 22:00.

C++ Programming. Disponível por www em: http://users.evtek.fi/~hannuvl/ke06/cplusplus_cap.htm, acesso em 10 de agosto de 2010 às 22:00.

CORMEN, Thomas H. **Algoritmos**: teoria e prática. 1ª ed. Rio de Janeiro: CAMPUS, 2002.

Curso de Linguagem C. Disponível por www em: http://ultradownloads.uol.com.br/download/Curso-de-Linguagem-C/, acesso em 10 de agosto de 2010, às 22:00.

Curso de Programação C++. Disponível por www em: http://s2i.das.ufsc.br/downloads/Curso_Programacao_Cplusplus_LCI.pdf, acesso em 10 de agosto de 2010, às 22:00.

Curso de Programação C da UFMG. Disponível por www em: http://www.descolando.com.br/resources/Curso_de_C_da_UFMG_1.pdf, acesso em 10 de agosto de 2010, às 22:00.

Curso de Programação em C++. Disponível por www em: http://www.ift.unesp.br/users/mmenezes/cpp/cpp.html, acesso em 10 de agosto de 2010, às 22:00.

De Objective Caml para C e C++/Os tipos básicos. Disponível

por www em: http://pt.wikibooks.org/wiki/De_Objective_Caml_para_C_e_C%2B%2B/Os_tipos_b%C3%A1sicos, acesso em 10 de agosto de 2010, às 22:00.

DEITEL, H. M. e DEITEL, P. J. **C++**: Como programar. Porto Alegre: 5ª ed. Bookman, Brasil, 2006.

Exercícios de programação II. Linguagem de Programação C. Disponível por www em: http://fit.faccat.br/~fpereira/apostilas/exercicios_c_mar2008.pdf, acesso em 10 de agosto de 2010, às 22:00.

FARRER, Cristiano G. Becker; FARIA, Eduardo; ET al. **Algoritmos Estruturados**. 3ª ed. São Paulo: LTC, 1999.

FRIGERI, Alceu Heinke; COPSTEIN, Bernardo; PEREIRA, Carlos Eduardo. **Curso de C++**. Porto Alegre, 1996.

GUIMARÃES, A. M. e LAGES, N. C. **Algoritmos e Estrutura de Dados**. LTC, 1994.

HARBISON III, Samuel P.; STEELE/JR., Guy; HARTMANN, Savannah. **C:** Manual de Referência. 1ª ed. Rio de Janeiro: Ciência Moderna, 2002.

JÚNIOR, Edwar Saliba. **Estruturas de Dados**. Notas de Aula. Faculdade de Tecnologia INED, Belo Horizonte, 2007.

KERNIGHAN, Brian. **The C Programming Language**, 2nd edition. Prentice Hall, 1988.

KNUTH, D. E. **The Art of Computer Programming**. Vol. 1. Fundamental Algoríthms. Addison Wesley, Reading, Mass., 1973.

MANZANO, J.A.N.G.. **Algoritmos**: lógica para desenvolvimento de programação de computadores. 1ª ed. São Paulo: Erica, 2002.

MIZRAHI, Victorine Viviane. **Treinamento em linguagem C**: módulos 1 e 2 : PLT.. 1ª ed. São Paulo: Pearson Prentice Hall, 2007.

Parâmetros interessantes do scanf e do printf em C. Disponível por www em: http://www.vivaolinux.com.br/artigo/Parametros-interessantes-do-scanf-e-do-printf-em-C/, acesso em 10 de agosto de 2010, às 22:00.

SCHILDT, Herbert. **C Completo e Total**. 3ª edição revista e atualizada. São Paulo: Makron Books, 1996.

SZWARCFITER, J. L. E MARKEZON, LILIAN. **Estrutura de dados e seus algoritmos**. Editora LTC. 1996.

TENEMBAUM, Aaron Ai; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. **Estruturas de Dados Usando C**. São Paulo: Makron Books, 1995.

VELOSO, Paulo, Et Alli: **Estrutura de Dados**. Editora Campos, 2a edição, 1984

ZIVIANI, Nivio. **Projeto de algoritmos com implementações Pascal e C**. 4. ed. São Paulo: Pioneira, 1999.

Minicurrículo

JOSÉ RICARDO MELLO VIANA



Possui graduação em bacharelado em Ciência da Computação pela Universidade Federal do Piauí (2006) e mestrado em Engenharia de Sistemas e Computação pela Universidade Federal do Rio de Janeiro (2009). Tem experiência na área de Ciência da Computação, com ênfase em Computação Gráfica e Programação, atuando principalmente nos seguintes temas: Algoritmos e Programação, Estruturas de Dados, Computação Gráfica, Programação em GPU, Programação para a Web, entre outros. Atualmente é professor efetivo da Universidade Federal do Piauí, lotado no Campus Senador Helvídio Nunes de Barros, em Picos.





www.uapi.ufpi.br